

MBA EM INTELIGÊNCIA ARTIFICIAL

Planejamento de Processos e Informação para Soluções com IA

Material consolidado das aulas: do mapeamento AS-IS ao PRD IA-first, passando por necessidades, indicadores e construção faseada.

18 aulas · 3 blocos

BLOCO 1 · AULA 01

Planejamento de Processos com enfoque IA-First (PRD)

Objetivos de Aprendizagem

Definir o que é um PRD (Product Requirements Document) e seu papel em projetos que envolvem IA.

Analisar e mapear um processo atual (AS-IS / SIS) identificando início, fim, gatilhos e responsáveis.

Identificar atritos em fluxos de processo e convertê-los em necessidades e requisitos.

Elaborar escopo, regras de negócio, personas e critérios de aceite para um produto/solução.

Priorizar necessidades usando categorias de prioridade e estruturar fases de entrega (SDD/ágeis).

Aplicar a metodologia para gerar um PRD pronto para ser consumido por uma IA ou equipe de TI.

Conceitos Principais

PRD — Documento de Requisitos do Produto

Definição: Documento técnico que descreve o que será construído, por que e como será avaliado (Product Requirements Document).

Finalidade: Servir de contrato entre a área solicitante, TI e a IA — reduz retrabalho, orienta desenvolvimento, define escopo e critérios de sucesso.

Formato: Objetivo e técnico (não precisa ser extenso), organizado em blocos (problema/objetivos, personas e escopo, necessidades/regras, solução to-be, dados/armazenamento, critérios de aceite).

Conceito IA-First

Abordagem que integra a IA desde a concepção do projeto, mas exige planejamento prévio: mapear processo, regras e requisitos antes de pedir à IA que construa.

Objetivo: Evitar "alucinações" da IA, reduzir iterações e economizar tokens/tempo.

SIS / AS-IS (Como está hoje)

SIS refere-se ao estado atual do processo: início, gatilho, etapas, responsáveis, entradas/saídas e fim.

Importante mapear com clareza para não automatizar falhas existentes.

To-Be (Solução desejada)

Descrição de como o processo deverá funcionar após a implementação: telas, regras automatizadas, integrações e persistência de dados.

Atritos e Necessidades

Atrito: ponto de dor no fluxo (ex.: solicitação por e-mail com dados incompletos).

Cada atrito identificado gera uma necessidade (o que deve ser resolvido).

Necessidades viram requisitos e soluções (ex.: formulário estruturado, validações, automações).

Escopo (In- / Out-)

Determina o que será incluído e explicitamente o que ficará de fora ("out") para evitar solicitação de funcionalidades já existentes ou fora do objetivo.

Importante para alinhamento com TI e stakeholders.

Regras de Negócio e Prioridades

Regras de negócio: restrições e permissões (quem pode criar tarefa, validações, exceções).

Prioridades: o que fazer agora, deve fazer, poderia fazer e não fazer (MVP / roadmap).

Fluxos, RACI e Handoffs

Fluxo: sequência de atividades com verbos (ex.: colaborador envia → gestor aprova → RH registra).

RACI: matriz que identifica quem é Responsável, Aprovador, Consultado e Informado.

Handoffs: trocas de responsabilidade entre atores (quem possui a tarefa em determinado momento).

Dados e Persistência

Definir fontes de dado (planilhas, banco de dados corporativo, APIs) e se o sistema deve armazenar resultados.

Critérios técnicos: tipo de armazenamento, segurança, local (on-prem / cloud).

Critérios de Aceite / Pronto

Critérios que definem quando uma funcionalidade/fase está concluída (ex.: ao clicar em aprovar, A, B e C acontecem e registro é gerado).

Podem existir critérios por fase (entregas incrementais).

Sequência de Trabalho recomendada (Fluxo de planejamento)

1. Selecionar o processo a ser mapeado (definir 5 etapas mínimas).
2. Mapear SIS/AS-IS: início, gatilhos, etapas, atores, fim.
3. Criar mapa mental do processo (centro: nome do processo; ramos: etapas/atores).
4. Converter mapa mental em fluxos detalhados (cada fluxo = verbo + ator).
5. Identificar atritos em cada fluxo.
6. Traduzir atritos em necessidades / requisitos.
7. Definir personas, escopo (in/out) e regras de negócio.
8. Priorizar necessidades (MVP / roadmap).
9. Projetar to-be: telas, integrações, persistências de dados.
10. Documentar critérios de aceite por entrega.
11. Gerar PRD (compilação técnica) e dividir em fases para construção por IA/equipe.

Aplicações e Exemplos Reais

Exemplo prático (aula em série): processo de solicitação de férias:

SIS: colaborador solicita → gestor aprova → RH processa → documento assinado.

Atrito: solicitação por e-mail com dados faltantes → necessidade: formulário padronizado com validações.

Solução to-be: formulário web com validações, regra: só gestor do time pode aprovar, armazenamento em DB corporativo, integração com folha de pagamento.

Exemplo logística (citado no texto):

Problema: registro inconsistente de erros de produção/atrasos.

Atrito: múltiplas fontes (papel, e-mail), dados incompletos.

Necessidade: formulário unificado + regras para classificação, integração com sistema de motoristas (se já existir, declarar como out).

Elementos Educacionais Detalhados

Mapas Mentais para Processos

Uso: organizar visualmente etapas e atores; facilitação para identificar fluxos e atritos.

Estrutura: centro = nome do processo; ramos = etapas/atores; sub-ramos = detalhes/entradas/saídas.

Fluxos e Verbos

Regra prática: cada linha do fluxo descreve um verbo (ação) executado por um ator.

Exemplo: "Colaborador → Submete formulário", "Gestor → Avalia", "RH → Armazena registro".

Como identificar Atritos eficazmente

Sinais: retrabalho, dados faltantes, tempo de espera, erros frequentes, baixa rastreabilidade.

Técnica: ao revisar cada linha do fluxo, perguntar "o que impede o progresso?" e anotar o atrito.

Priorização (MVP)

Categoria simples: Agora (crítico), Deve (importante), Poderia (opcional), Não (fora do escopo atual).

Justifique prioridades com impacto e complexidade.

Terminologia Técnica

Termo	Definição	Contexto / Exemplo
PRD	Product Requirements Document — documento de requisitos do produto	Documento que descreve problema, objetivos, personas, escopo, regras, dados e critérios de aceite.
SIS / AS-IS	Estado atual do sistema/processo	Mapeamento de como o processo funciona hoje (gatilho, etapas, responsáveis).
To-Be	Estado futuro desejado do processo/sistema	Descrição da solução após implementação.
Atrito	Ponto de dor ou fricção no fluxo	Ex.: solicitação por e-mail sem campos obrigatórios.
RACI	Matriz de responsabilidade (Responsible, Accountable, Consulted, Informed)	Define quem faz, aprova, consulta e é informado.
Handoff	Transferência de responsabilidade entre pessoas/áreas	Ex.: colaborador envia → gestor assume até aprovação.
Critério de Aceite	Requisitos objetivos que definem "pronto"	Ex.: botão aprovar gera registro, notifica RH e atualiza dashboard.
Persistência de Dados	Onde/como os dados serão armazenados	Planilha, banco de dados SQL, Data Lake, etc.
IA-First	Estratégia que considera IA desde o planejamento	PLANEJAR antes de pedir que IA desenvolva para evitar alucinações.

Pontos de Atenção e Equívocos Comuns

Não automatize processos que estão com falhas graves sem antes corrigi-los: você só transfere o problema.

Evite pedir à IA construir sem PRD: resultados abstratos, alto retrabalho e custos.

Não inclua tudo no escopo inicial; delimite claramente o que fica de fora.

Priorize por impacto e viabilidade técnica — não por desejo de recursos extras.

Regras de negócio devem ser validadas com áreas responsáveis (TI, compliance, jurídico quando aplicável).

Atividades Práticas e Questões de Revisão

Exercício prático (individual):

1. Escolha um processo da sua área (ex.: aprovação de despesas, onboarding de funcionário, gestão de chamados).
2. Liste 5 etapas mínimas desse processo (início → fim).
3. Crie um mapa mental simples (pode ser desenhado) com o nome do processo no centro e ramos para cada etapa.
4. Para cada etapa, descreva o ator responsável e identifique ao menos um atrito.
5. Converta cada atrito em uma necessidade (requisito).
6. Priorize as necessidades em Agora / Deve / Poderia / Não.
7. Esboce rapidamente um critério de aceite para a entrega "Agora".

Perguntas de revisão (níveis variados):

Compreensão: O que é um PRD e por que ele é essencial em projetos com IA?

Aplicação: Dê um exemplo de um atrito em um processo de compras e descreva a solução to-be.

Análise: Como o mapeamento SIS pode impactar a qualidade da solução final?

Avaliação: Quais critérios você usaria para decidir se um requisito deve entrar no MVP?

Síntese: Monte, em até 10 linhas, um parágrafo do PRD que descreva o problema e objetivo de uma solução de controle de ponto.

Discussão em grupo:

Debata riscos de automatizar processos sem corrigir erros estruturais. Traga exemplos da sua experiência.

Como balancear rapidez de entrega com necessidade de documentação técnica (PRD) em times com pressão por resultado?

Resumo (Pontos-chave)

Sempre inicie por mapear o processo atual (SIS/AS-IS) antes de pedir que IA desenvolva.

Identifique atritos; cada atrito justifica uma necessidade e, portanto, um requisito.

Um PRD bem-feito é curto, técnico e contém: problema/objetivos, personas e escopo, necessidades e regras, solução to-be, dados e critérios de aceite.

Priorize e entregue em fases; não entregue o PRD inteiro de uma vez para construção sem divisão em fases.

Defina claramente o que fica de fora (out) para alinhar expectativas com TI e stakeholders.

Use mapas mentais e fluxos verbais para transformar visões gerais em instruções práticas.

BLOCO 1 · AULA 02

Mapeamento de Processos: Exemplo Prático — Solicitação de Férias (Aula 2)

Objetivos de Aprendizagem

Definir o conceito de As-Is e entender sua importância no mapeamento de processos.
 Identificar e descrever as etapas de um processo operacional (solicitação de férias) usando mapa mental e fluxos.
 Aplicar regras de escopo (início, fim, fora do escopo) para delimitar um processo.
 Traduzir um mapa mental em um fluxo As-Is com atividades escritas em verbo.
 Detectar pontos de atrito, atores envolvidos e fontes/sistemas utilizados no processo atual.
 Preparar o material inicial (documentação) para posterior análise de melhorias e geração de PRD.

Conceitos-chave

1. As-Is (Estado Atual)

Definição: Descrição de como o processo funciona hoje, sem propor soluções.
 Objetivo: Entender o funcionamento atual para evitar automatizar desperdícios e revelar regras ocultas ou exceções.
 Aplicação: Usado como base para contar a história do processo para IA ou para preparar um PRD.

2. Escopo do Processo

Início: Evento que dispara o processo (gatilho).
 Fim: Resultado esperado que encerra o processo mapeado.

Fora do Escopo: O que não será abordado no mapeamento/sistema atual (ajuda a evitar projetos inchados).

Atores: Pessoas ou sistemas que participam do processo (ex.: Colaborador, Gestor, DP).

Sistemas/Fonte: Meios usados hoje (ex.: e-mail, sistema de DP).

3. Mapa Mental

Ferramenta para visualizar o processo como um todo (zoom out).

Ajuda a estruturar etapas e decisões antes de "linearizar" em um fluxo.

Facilita identificar regras, exceções e pontos que geram dúvidas.

4. Fluxo As-Is

Tradução do mapa mental em uma lista linear de atividades.

Cada linha/etapa deve começar com um verbo (ex.: "Solicitar", "Aprovar", "Enviar e-mail").

Deve indicar quem executa cada atividade, qual a fonte/sistema e onde começa/termina.

Exemplo Prático: Processo de Solicitação de Férias (Estado Atual)

Contexto: Empresa onde hoje o processo é majoritariamente por e-mail e papel. O objetivo do exercício é mapear o fluxo atual (As-Is) desde a solicitação até a assinatura dos documentos.

Escopo inicial:

Nome do processo: Solicitação de férias

Objetivo do mapeamento: Entender como as férias são solicitadas e quais aprovações são necessárias.

Início (gatilho): Colaborador solicita férias para o gestor.

Fim (resultado): Colaborador assina o recibo/termo de férias (documento entregue pelo DP).

Fora do escopo: Confirmação prévia com a equipe (decisão de incluir essa etapa depende do projeto).

Atores principais: Colaborador, Gestor, Departamento Pessoal (DP).

Sistemas/fonte atual: E-mail (principalmente); possíveis papéis físicos.

Fluxo As-Is (etapas identificadas)

1. Colaborador solicita férias para o gestor.
2. Gestor analisa e decide aprovar ou não aprovar.
 - Se aprovado:
3. Gestor envia e-mail ao colaborador com cópia para o DP autorizando as férias.
4. DP cadastra a informação no sistema de DP da empresa.
5. DP imprime o recibo/termo de férias e chama o colaborador para assinar.
6. Colaborador assina o documento — fim do processo.
 - Se não aprovado:
7. Gestor envia e-mail ao colaborador negando ou solicitando mais informações — fim do processo.

Observações levantadas durante mapeamento:

Há risco/pressuposto de que o colaborador já confirmou com sua equipe; decidir se isso será formalizado (fora do escopo ou exigência no sistema).

Necessidade de notificação do DP ao colaborador para que este compareça para assinar (atividade que pode ter sido omitida inicialmente).

Pontos potenciais de atrito: atrasos nas respostas do gestor, ausência de formalização da conversa com a equipe, tempo entre aprovação e assinatura.

Termos Técnicos e Definições

Termo	Definição	Contexto/Exemplo
As-Is	Descrição do estado atual de um processo	Mapear como solicitações de férias ocorrem hoje (e-mail + papel)
Escopo	Limites do que será incluído no mapeamento	Início: solicitação ao gestor. Fora do escopo: conversa prévia com equipe (opcional)
Mapa Mental	Representação visual não-linear do processo	Usado para desenhar árvore de etapas e decisões antes de linearizar
Fluxo As-Is	Versão linear do processo, em etapas com verbos	"Colaborador solicita", "Gestor aprova", "DP cadastra"
Ator	Pessoa ou sistema que executa uma etapa	Colaborador, Gestor, DP, Sistema de DP
Fonte/Sistema	Meio pelo qual a atividade é feita	E-mail, sistema de RH, papel impresso
Atrito (gargalo)	Ponto com problemas ou perdas de eficiência	Tempo de resposta do gestor, falta de aviso do DP ao colaborador
PRD (Product Requirements Document)	Documento que resume o que foi mapeado para orientar a solução	PRD recebe resumo do As-Is e necessidades, não descreve cada detalhe

Resumo (Pontos Principais)

Comece descrevendo o As-Is; não proponha soluções nesta etapa.
 Use mapa mental para ter visão global e capturar decisões/variantes.
 Delimite escopo claramente (início, fim, fora do escopo, atores).
 Converta o mapa mental em um fluxo As-Is com atividades iniciadas por verbos.
 Cada etapa do fluxo deve indicar quem executa e qual a fonte/sistema.
 Documente problemas e atritos identificados durante o mapeamento.
 Faça backup dos dados do seu planejador (ex.: exportar JSON) para evitar perda.

Perguntas de Revisão e Exercícios Práticos

Compreensão (nível básico)

1. O que significa As-Is e por que é importante antes de propor soluções?
2. Quais são os elementos mínimos que devem constar no escopo de um processo?

Aplicação (nível intermediário)

3. A partir do exemplo apresentado, reescreva o fluxo As-Is incluindo uma etapa onde o colaborador registra no sistema que conversou com sua equipe (decida se entra no escopo ou permanece fora). Explique sua escolha.
4. Identifique ao menos três potenciais gargalos nesse processo e proponha uma métrica para medir cada um (ex.: tempo médio entre solicitação e aprovação).

Análise (nível avançado)

5. Suponha que o gestor frequentemente demora 72 horas para aprovar solicitações. Quais dados você coletaria durante o mapeamento para justificar uma melhoria? Como você apresentaria isso no PRD?
6. Explique como um mapa mental pode ajudar a revelar regras ocultas e exceções que um simples fluxo linear poderia ocultar.

Discussão / Reflexão

7. Que vantagens e riscos existem ao incluir a validação prévia com a equipe diretamente no sistema (como etapa formal)?
8. Quando é correto manter partes do processo “fora do escopo”? Quais critérios você usaria para decidir isso?

Atividade prática

Pegue um processo simples do seu trabalho (ex.: pedido de reembolso, solicitação de compra, aprovação de conteúdo) e:

1. Preencha Início, Fim, Atores, Sistemas e Fora do Escopo.
2. Desenhe um mapa mental com as etapas e decisões.
3. Converta o mapa em um fluxo As-Is com cada linha começando por um verbo.

4. Identifique ao menos dois pontos de atrito e um dado que você poderia coletar para quantificar cada atrito.

Possíveis Dúvidas e Armadilhas Comuns

"Já que vamos automatizar, por que mapear o As-Is?" — Se você automatiza sem diagnóstico, você digitaliza a bagunça. O As-Is revela regras e exceções que guiam a solução correta.

"Tenho dificuldade em transformar ideias em verbos." — Pense nas ações realizadas: enviar, aprovar, assinar, cadastrar, imprimir. Cada etapa do fluxo deve ser uma ação observável.

"Escopo muito amplo travou o projeto." — Use explicitamente o "Fora do Escopo" para cortar aquilo que não será tratado agora.

"Esqueci de uma etapa depois de converter para fluxo." — Volte ao mapa mental e atualize; o processo é iterativo.

Estratégias para Superar Desafios no Mapeamento

Faça o exercício ao vivo: enquanto assiste a uma explicação, preencha já o mapa mental no seu computador ou papel.

Entrevistas com quem executa o processo (evidências reais) são essenciais; prepare checklist e perguntas diretas.

Mantenha iterações curtas: mapear, revisar, ajustar; preferencialmente em par com alguém que conheça o processo.

Backup frequente dos seus arquivos (exportar JSON / salvar localmente) para evitar perda de trabalho.

BLOCO 1 · AULA 03

Mapeamento de Atritos em Processos: Guia Prático para Diagnóstico e Transformação

Objetivos de Aprendizagem

Definir o que são atritos em processos e distinguir entre sintoma e causa.

Analisar um mapa As-Is para identificar atritos reais em cada etapa do fluxo.

Classificar tipos de atritos (retrabalho, espera, informação, decisão) e avaliar impacto e frequência.

Formular necessidades a partir de atritos evidenciados e relacioná-las a soluções (incluindo automação com IA).

Aplicar método estruturado para registrar evidências, priorizar atritos e transformar diagnósticos em requisitos práticos.

Conceitos-chave

1. O que são Atritos?

Definição: Atritos são problemas, obstáculos ou pontos de “dor” identificáveis dentro de um processo que impedem a operação fluida, provocam retrabalho, atrasos ou decisões inadequadas.

Exemplo prático: Colaborador solicita férias por e-mail sem informar datas completas — gera troca de mensagens, atraso e retrabalho.

2. Sintoma vs Causa

Sintoma: Manifestação observável (por ex.: “demora muito”).

Causa: Motivo raiz que gera o sintoma (por ex.: aprovação depende de e-mail e não há SLA).

Regra prática: Evite tratar apenas sintomas (solução cosmética). Busque causas para definir necessidades reais.

3. Fluxo As-Is e Diagnóstico

Mapeamento As-Is: Primeiro passo — descrever passo a passo como o processo acontece hoje.

Diagnóstico: Ler o mapa As-Is procurando por atritos reais em cada atividade.

Handoff: Indica responsabilidade entre etapas (importante para identificar onde o atrito realmente pertence).

4. Tipos de Atrito

Retrabalho: A mesma atividade precisa ser refeita (ex.: preencher formulário novamente).

Espera: Tempo de inatividade aguardando resposta ou ação (ex.: gestor demora a aprovar).

Informação: Dados incompletos ou incorretos (ex.: falta data de início/retorno).

Decisão: Falta de critérios, alçada concentrada, gargalo humano (ex.: decisão só pode ser tomada por um gestor sobrecarregado).

5. Características que Devem Ser Registradas para Cada Atrito

Onde ocorre (etapa do fluxo).

Tipo (retrabalho, espera, informação, decisão).

Impacto (alto, médio, baixo).

Frequência (sempre, às vezes, raramente).

Evidência (ex.: e-mails, logs, registros).

Observações e possíveis regras relacionadas (ex.: antecedência mínima para solicitar férias).

6. Transformação: Atrito → Necessidade → Solução

Cada atrito deve gerar, quando pertinente, uma necessidade (requisito).

Necessidades servem de base para desenvolver soluções (funcionais, automação com IA, regras de negócio).

Exemplo: Atrito “falta de data nas solicitações” → Necessidade: campo obrigatório data de início/fim no formulário → Solução: ajuste no formulário + validação.

7. Papel da IA no Processo

IA como facilitadora: pode implementar correções rápidas (ex.: pequenas mudanças de UI, validações, geração de campos).

Recomenda-se registrar também atritos de baixo volume quando o esforço de correção via IA for pequeno e impacto alto em situações pontuais.

Cuidado: nem todo atrito justifica automação; é necessário diagnóstico.

Termos Técnicos

Termo	Definição	Contexto
As-Is	Mapeamento do estado atual do processo	Base para identificar atritos e melhorar processos
Atrito	Problema identificado numa etapa do processo	Pode causar atraso, retrabalho, erro ou decisão indevida
Sintoma	Indicação observável do problema (efeito)	Ex.: demora, aumento de e-mails, reclamações
Causa	Raiz do problema que gera o sintoma	Necessária para soluções duradouras
Necessidade	Requisito gerado a partir de um atrito	Base para o desenvolvimento de soluções
SLA	Acordo de nível de serviço (prazos)	Usado para controlar tempos de resposta/decisão
Handoff	Transição de responsabilidade entre etapas	Importante para localizar o dono do atrito
IA (Inteligência Artificial)	Ferramenta para automatizar/implementar mudanças	Útil para correções rápidas e geração de funcionalidades

Guia Passo a Passo para Identificação e Registro de Atritos

1. Mapear o fluxo As-Is por atividades/etapas.
2. Para cada atividade, perguntar: há retrabalho, espera, falta de informação ou decisão concentrada?
3. Cadastrar cada atrito com: etapa, tipo, descrição curta, impacto, frequência e evidências.
4. Priorizar atritos por impacto e frequência (e facilidade de correção via IA).
5. Converter cada atrito relevante em uma necessidade — descrever o requisito de solução.
6. Agrupar necessidades no mapa mental de necessidades (ligadas aos atritos).
7. Não apagar atritos sem checar relações com necessidades (aviso técnico no sistema).
8. Antes de automatizar, diagnosticar causa — não automatizar desperdício.
9. Usar IA para pequenas correções quando custo-benefício favorável (tempo e esforço baixos, impacto relevante).
10. Revisar com stakeholders e ajustar prioridades.

Exemplos e Estudos de Caso (Aplicações Reais no Contexto da Aula)

Exemplo 1 — Solicitação de Férias:

Atrito: Colaborador envia e-mail sem data de início/fim.

Impacto: Alto (retorno de mensagens, atraso de aprovação).

Frequência: Às vezes.

Evidência: Trocas de e-mail com pedido de informação adicional.

Necessidade: Formulário com campos obrigatórios para data de início e de retorno; validação e checagem automática.

Solução via IA/UI: Campo obrigatório + validação no front-end; notificação automática ao gestor.

Exemplo 2 — Aprovação Demorada:

Atrito: Gestor demora mais de X dias para aprovar férias.

Impacto: Médio/Alto (colaborador sem resposta, reorganização de equipe).

Evidência: Logs de tempo entre solicitação e aprovação.

Necessidade: SLA de aprovação; alertas automáticos e escalonamento.

Solução: Implementar regras de SLA, alertas via sistema e possível delegação de alçada para casos simples.

Exemplo 3 — Faltou Antecedência:

Atrito: Colaborador solicita sem antecedência mínima (ex.: 15 dias).

Impacto: Alto (gestor não consegue planejar).

Evidência: Datas de solicitação vs. data de início.

Necessidade: Regra de negócio que bloqueia solicitação com antecedência insuficiente + justificativa para exceções.

Solução: Validar data no formulário; campo de justificativa e fluxo de exceção manual.

Pontos de Atenção e Boas Práticas

Registre evidências (fatos) — não baseie diagnósticos em “achismos”.

Se estiver em dúvida sobre cadastrar um atrito, registre-o; IA pode facilitar correções.

Não automatize processos sem entender a causa raiz.

Para cada atrito, identifique o responsável (handoff) para evitar confusão sobre onde atuar.

Priorize atritos com alto impacto e frequência; no entanto, se a correção for barata e rápida, trate também atritos esporádicos de alto impacto.

Mantenha rastreabilidade entre atrito → necessidade → solução.

Resumo (Pontos Finais)

Atritos são a origem das necessidades de mudança: mapear atritos corretamente é fundamental para soluções eficazes.

Sintoma sem causa gera solução superficial; busque sempre a raiz.

Classifique e registre atritos com impacto, frequência e evidência.

Transforme atritos em necessidades e utilize IA quando for eficiente para correção.

A prática (mão na massa) é essencial: experimente cadastrar pelo menos 3–5 atritos no seu processo para treinar.

Perguntas de Revisão e Exercícios

1. Defina "atrito" e dê três exemplos que poderiam ocorrer num processo de requisição de férias.
2. Explique a diferença entre sintoma e causa com um exemplo prático.
3. Para uma etapa em que "o gestor demora para aprovar", liste quais informações você registraria ao cadastrar esse atrito.
4. Imagine que o colaborador frequentemente envia solicitações sem quantidade de dias. Descreva a necessidade e proponha uma solução técnica.
5. Discussão: Em que situações não vale a pena automatizar um atrito? Quais critérios usar para decidir?
6. Exercício prático: Pause a aula e cadastre 3 atritos reais (ou simulados) de um processo que você conheça. Para cada atrito, registre tipo, impacto, frequência e evidência.

BLOCO 1 · AULA 04

Regras de Negócio (mapeamento e documentação)

Objetivos de Aprendizagem

Definir o que é uma regra de negócio e distinguir regras de preferências ou requisitos estéticos.

Analisar e classificar regras de negócio por tipo (elegibilidade, cálculo, validação, SLA, alçada).

Aplicar a metodologia de mapeamento de regras vinculadas a fluxos e etapas do processo (As-Is).

Documentar regras de forma testável, auditável e pronta para uso em soluções com IA.

Avaliar se uma regra deve ser mantida, adaptada ou descartada ao migrar para a solução (To-Be).

Conceitos Principais

O que é uma regra de negócio

Uma regra de negócio é uma lógica que determina decisões dentro de um processo. Geralmente tem a forma condicional: "Se [condição], então [ação]".

Deve ser testável (é possível verificar sua aplicação), auditável (tem origem ou justificativa documentada) e mensurável/identificável.

Exemplo prático: "Se a diferença entre a data de solicitação e a data de início das férias for menor que 15 dias, bloquear o envio."

Diferença entre regra, preferência e necessidade estética

Regra de negócio: determina comportamentos e validações (ex.: prazos, cálculos, autorizações).

Preferência/estética: escolhas de UI/UX (ex.: botão azul, tela moderna) que são necessidades de solução, não regras de negócio. Podem virar requisitos funcionais, mas não regras lógicas testáveis.

Necessidade: nasce de um atrito (dor) identificado e pode virar solução (ex.: tornar a tela mais clara para reduzir confusões).

Vínculo das regras com fluxo e etapa

Toda regra normalmente está atrelada a uma etapa específica do fluxo (por exemplo: "Colaborador solicita férias → Gestor aprova → DP processa").

Ao mapear um fluxo, para cada decisão/etapa devemos checar: existe alguma regra que define ou limita essa ação?

Nem toda regra exige criar um "atrito" novo; pode existir sem estar atrelada a um atrito, simplesmente porque faz parte do processo.

Tipos de regras (exemplos)

Elegibilidade: quem pode ou não executar determinada ação (ex.: tempo mínimo de casa).

Cálculo: fórmulas para saldo, proporções, prazos (ex.: diferença entre datas > 15 dias).

Validação: formatos, dados obrigatórios, consistência (ex.: indicar se conversou com a equipe).

Alçada/autorização: quem aprova até que valor/condição.

SLA / prazos operacionais: tempo máximo para aprovação, envio ao DP, resposta do colaborador.

Origem e rastreabilidade

Sempre indicar de onde vem a regra: política interna, legislação (CLT), entrevista com stakeholders, planilha, e-mail, etc.

Documentar exceções e pendências. Exceções não documentadas viram comportamentos-surpresa.

Status de uma regra: confirmada (válida hoje), suspeita (precisa confirmar), descartada, adaptada.

Regra testável e critério de aceite

Regra deve ser mensurável para permitir critério de aceite. Frases vagas como "gestor deve aprovar rápido" não são aceitáveis.

Regra bem formulada: "Aprovar ou recusar em até 2 dias úteis; se não, escalar para o RH."

Metodologia de Mapeamento (passo a passo)

1. Mapear o fluxo As-Is (etapas e atritos identificados).
2. Para cada etapa: identificar decisões e perguntar "com base em que se decide isso?"
3. Transformar decisões em regras no formato condicional (SE... ENTÃO...).
4. Classificar a regra (tipo, origem) e indicar se é testável.
5. Registrar exemplos válidos e inválidos para clarificar a aplicação.
6. Anotar exceções e conflitos; marcar pendências de alinhamento quando necessário.
7. Agrupar regras que são gerais do sistema (ex.: fuso horário padronizado) e regras específicas de fluxo.
8. Ao migrar para o To-Be / solução, decidir por cada regra: manter, adaptar ou descartar. Indicar o destino de uso (regra geral do sistema ou vinculada à solução).

Exemplos práticos retirados do caso de férias (aplicação)

1. Regras de validação (vínculo: colaborador solicita)
 - Regra: "Colaborador deve indicar se falou com a equipe; se não indicar, não deve conseguir enviar a solicitação."
 - Tipo: validação / elegibilidade
 - Testável: sim (campo obrigatório/flag)
 - Exemplo válido: colaborador marca "Sim, confirmei com a equipe"
 - Exemplo inválido: colaborador marca "Não" ou não marca nada
2. Regra de prazo mínimo para solicitação
 - Regra: "Solicitação só é permitida se a data de solicitação for anterior à data de início das férias por, no mínimo, 15 dias."
 - Tipo: cálculo de data / validação
 - Testável: sim (diferença entre datas ≥ 15)
 - Exemplo inválido: pedir em 2/1 para férias que começam em 15/1 (diferença < 15)
3. Regra de prazo para envio do gestor ao DP
 - Regra: "Se a aprovação do gestor for enviada com menos de 10 dias para o início das férias, o DP deve recusar (ou o sistema deve bloquear)."
 - Tipo: SLA / validação
 - Origem: política interna / requisito operacional
 - Exemplo inválido: gestor envia aprovação em 5/1 para férias começando em 11/1

4. Regras de comunicação e resposta

- Regra: "Se reprovado, gestor deve enviar e-mail com motivo e prazo para o colaborador responder (ex.: 5 dias)."
- Tipo: processo / SLA

Terminologia Técnica

Termo	Definição	Contexto
Regra de negócio	Logica condicional que orienta decisões dentro de um processo.	Usada para validação, cálculo, elegibilidade, SLA.
As-Is	Estado atual do processo mapeado.	Base para identificar regras e atritos existentes.
To-Be	Estado desejado / solução futura que implementará regras selecionadas.	Onde regras são mantidas, adaptadas ou descartadas.
Atrito (dor)	Problema ou dificuldade identificada no processo que gera necessidade de solução.	Pode originar necessidades e soluções; nem todo atrito vira regra.
Testável	Propriedade de uma regra que permite verificá-la por meio de testes.	Essencial para critérios de aceite.
Exceção	Caso que foge à regra padrão e deve ser documentado.	Exceções não documentadas causam comportamentos-surpresa.
SLA	Acordo de nível de serviço, geralmente tempo máximo para execução.	Ex.: tempo para gestor aprovar ou para DP processar.

Pontos de Atenção e Dificuldades Comuns

Vagueza: regras escritas de forma ambígua (ex.: "aprovar rápido") não são úteis. Converter em tempos, condições e ações claras.

Confusão entre regra e preferência estética: manter distinção para não inflar o conjunto de regras.

Regras não testáveis: se não é possível verificar, não é regra; pode ser critério da etapa ou necessidade de mais informações.

Conflitos entre regras: documentar e marcar como pendência para alinhamento, priorizando origem (lei, política, prática corrente).

Documentar exceções: essencial para evitar resultados inesperados.

Sumário (Principais Lições)

Regra de negócio é condicional, testável e auditável.

Regras devem ser mapeadas vinculadas a fluxos/etapas (As-Is) e depois decididas para o To-Be.

Classificar origem, tipo, testabilidade e exemplos torna a regra operacional e implementável.

Nem toda necessidade é regra; preferências e exigências estéticas devem ser tratadas separadamente.

Documente conflitos, pendências e o destino de cada regra (manter, adaptar, descartar).

Questões de Revisão (níveis variados)

Compreensão

1. Defina com suas palavras o que é uma regra de negócio e dê um exemplo relacionado a férias.
2. Qual a diferença entre uma regra de negócio e uma preferência de interface?

Aplicação

3. Dado: colaborador solicita férias em 10/08; início previsto em 20/08; prazo mínimo é 15 dias. O sistema deve permitir? Explique o cálculo.
4. Redija a regra condicional (SE... ENTÃO...) para obrigar o colaborador a confirmar que conversou com a equipe antes de enviar a solicitação.

Análise / Avaliação

5. Você encontrou duas regras conflitantes: Regra A exige aviso mínimo de 15 dias; Regra B exige que o gestor envie ao DP com pelo menos 10 dias. Como você documenta e resolve esse conflito?
6. Como você decide se uma regra mapeada no As-Is deve ser mantida, adaptada ou descartada no To-Be?

Discussão / Projeto

7. Proponha um pequeno formulário de solicitação de férias que incorpore 4 regras testáveis (liste campos e validações).
8. Discuta como tornar as regras documentadas "prontas para IA" — o que é necessário para que um modelo de IA consiga gerar código ou especificações com base nessas regras?

Pré-requisitos

Noções básicas de mapeamento de processos (fluxos).

Entendimento de conceitos de produto mínimo viável (MVP) e priorização.

Conhecimentos básicos sobre prazos, cálculos de datas e documentação de requisitos.

Dicas Práticas

Ao mapear, escreva a regra em linguagem condicional clara (SE... ENTÃO... SENÃO...).

Sempre inclua exemplos válidos e inválidos para cada regra.

Marque o status e a origem da regra para facilitar rastreabilidade.

Mantenha o foco no essencial (MVP): poucas regras bem definidas são melhores que muitas vagas.

Crie um repositório único (planilha ou ferramenta) para registrar regras, origem, status e destino (To-Be).

BLOCO 1 · AULA 05

Mapeamento de Responsabilidades e Handoffs (RACI/RACE) aplicado a processo de solicitação de férias

Objetivos de Aprendizagem

Definir e diferenciar os papéis R (Responsible), A (Accountable), C (Consulted) e I (Informed) no contexto de mapeamento de processos.

Aplicar a matriz RACI/RACE para cada etapa de um fluxo de trabalho (ex.: solicitação de férias).

Identificar e mapear handoffs (passagens de bastão) entre atores do processo.

Diagnosticar riscos e atritos que surgem em handoffs e registrar evidências para gerar necessidades e soluções.

Planejar ações para reduzir atrasos e retrabalho derivados de handoffs mal definidos.

Conceitos-chave

1. Importância de definir o "dono" do processo

Explicação: Sem um dono claro, a melhoria falha porque não há responsabilidade definida sobre executar, cobrar e garantir resultado.

Aplicação real: Ao mapear processos para automação, certifique-se de que o processo já existe e que o dono está identificado para viabilizar a automação.

2. Matriz RACE / RACI (Responsible, Accountable, Consulted, Informed)

Responsible (R): Quem executa a atividade, quem faz o trabalho.

Accountable (A): Quem presta contas pelo resultado; há somente um A por atividade.

Consulted (C): Quem deve ser consultado antes da execução (tem voz/opinião).

Informed (I): Quem é apenas informado do que aconteceu (não opina).

Boas práticas:

Cada atividade deve ter pelo menos um R e exatamente um A.

Evitar múltiplos A para a mesma atividade (caso contrário, modelar como atividades separadas).

Evitar consultar demasiadas pessoas (decisão lenta).

Exemplo aplicado (fluxo de férias):

Atividade: "Colaborador solicita férias para o gestor"

R: Colaborador (executa a solicitação)

A: Colaborador (presta contas pelo envio correto)

C: (opcional) Equipe do colaborador

I: Gestor (recebe aviso)

Atividade: "Gestor envia e-mail autorizando com cópia para DP"

R: Gestor

A: Gestor

I: DP, Colaborador

3. Handoff (Passagem de bastão)

Definição: Momento em que o trabalho sai de uma pessoa e vai para outra.

Porque é crítico: Handoffs são onde ocorrem mais atrasos, perda de informação e retrabalho.

Elementos a mapear em um handoff:

Quem transfere (origem) e quem recebe (destino).

Entre quais atividades do fluxo ocorre a mudança.

O que é transferido (conteúdo/informação/artefato).

Formato do envio (e-mail, sistema, físico).

Gravidade caso falhe (alto, médio, baixo).

O que costuma falhar (exemplos e evidências).

Resultado do mapeamento: identificação de atritos (riscos) que podem originar necessidades e, posteriormente, soluções.

4. Atritos, Riscos e Necessidades

Atrito: Dor ou falha que acontece, frequentemente relacionada a um handoff.

Risco: Probabilidade/impacto de uma falha acontecer.

Necessidade: Demanda gerada a partir de um atrito identificado; base para justificar soluções/produtos.

Evidência: Ex.: e-mails trocados, número de ocorrências; importante para priorizar necessidades.

Passo a passo prático (sequência sugerida)

1. Mapear o fluxo AS-IS (atividades em sequência – verbs/ações).
2. Identificar atores principais e cadastrá-los (colaborador, gestor, DP, equipe, etc.).
3. Para cada atividade:
 - Preencher R (Responsible) e A (Accountable) obrigatoriamente.
 - Avaliar se há C (Consulted) e/ou I (Informed).
4. Identificar handoffs quando o R muda de uma atividade para outra.
5. Para cada handoff:
 - Descrever o que é transferido e em qual formato.
 - Classificar gravidade caso falhe.
 - Documentar o que costuma falhar e registrar evidências.
6. Registrar atritos/riscos vinculados ao handoff ou à etapa.
7. Priorizar necessidades a partir dos atritos com maior impacto/frequência.
8. Planejar soluções (PRD/Backlog) a partir das necessidades validadas.

Termos Técnicos

Termo	Definição	Contexto / Exemplo
RACI / RACE	Matriz de papéis: Responsible, Accountable, Consulted, Informed	Usada para clarificar responsabilidades em cada atividade de processo.
Responsible (R)	Executa a tarefa	Colaborador que preenche e envia solicitação de férias.
Accountable (A)	Presta contas pelo resultado; única pessoa que aprova	Gestor que aprova ou reprovava férias; por atividade só um A.
Consulted (C)	Parte consultada antes da ação (participação ativa)	Equipe do colaborador consultada sobre datas.
Informed (I)	Apenas informada do resultado (sem participação ativa)	DP informado sobre aprovação por e-mail.
Handoff	Passagem de bastão entre atores/atividades	Do colaborador para o gestor quando solicita as férias.
Atrito	Falha/dor identificada em uma etapa ou handoff	Falta de informações no e-mail que causa retrabalho.
Necessidade	Demanda de melhoria/solução gerada por atrito	Sistema que padronize envio e registro de solicitações.
Evidência	Prova de ocorrência do atrito	Threads de e-mail, contagem mensal de falhas.

Pontos de Atenção / Possíveis Confusões

Um único Accountable (A) por atividade: se mais de uma pessoa precisa aprovar, modele atividades distintas (ex.: aprovação do coordenador + aprovação do gestor = duas etapas).

R e A podem ser a mesma pessoa (comum em solicitações individuais), mas isso deve ser explícito.

Consultar muitas pessoas diminui velocidade decisória — balanceie necessidade de consulta vs. agilidade.

Handoffs nem sempre óbvios: verifique mudanças de R entre fases para identificar handoffs ocultos.

Atrito não é backlog: registrar atrito é diferente de automaticamente gerar uma tarefa de desenvolvimento; atrito gera necessidade que deve ser priorizada.

Exemplos e Estudo de Caso (Solicitação de Férias)

Fluxo resumido (exemplo prático):

1. Colaborador solicita férias ao gestor.
2. Gestor aprova/nega e envia e-mail com cópia ao DP.
3. DP cadastra no sistema.
4. DP imprime recibo e chama colaborador para assinar o documento.
5. Colaborador assina e confirma recebimento (ou confirma recebimento de negativa).

Principais handoffs mapeados:

Handoff 1: Colaborador -> Gestor (solicitação enviada por e-mail)

Risco: colaborador esquece data/quantidade de dias (falta de informação).

Impacto: alto.

Handoff 2: Gestor -> DP (envio de aprovação com cópia)

Risco: e-mail do DP administrado por várias pessoas; pode ficar perdido.

Impacto: alto.

Handoff 3: DP -> Colaborador (documento físico para assinatura)

Risco: colaborador não comparece/assinatura atrasada.

Impacto: alto.

Handoff 4 (negativa): Gestor -> Colaborador (aviso de negativa)

Risco: colaborador não confirma recebimento; gestor fica sem prova.

Impacto: médio.

Exemplo de atrito registrado:

Atrito: "E-mail do DP é administrado por mais de um colaborador e a solicitação fica perdida."

Evidência: ocorrência de X em data Y; 3 erros no mês.

Impacto: alto → gera necessidade de solução (ex.: caixa com responsável único, sistema com rastreabilidade).

Resumo dos Principais Pontos

Definir claramente quem faz e quem responde em cada atividade (R e A).

Identificar e documentar todos os handoffs; estes são os pontos mais críticos do processo.

Registrar atritos com evidências para justificar necessidades de solução.

Um A por atividade evita ambiguidade; múltiplos executores (R) são possíveis, mas aprovação deve ser única.

Mapeamento gera insights: muitas vezes ao mapear handoffs surgem etapas faltantes ou atividades que precisam ser refinadas.

Perguntas de Revisão e Exercícios

Compreensão:

1. Explique a diferença entre Responsible e Accountable com exemplos.
2. Por que é importante ter somente um Accountable por atividade?

Aplicação:

3. Dado um fluxo hipotético "Requisição de compra", identifique possíveis handoffs e pelo menos dois atritos que podem surgir.
4. No fluxo de férias descrito, descreva uma solução técnica que poderia reduzir o risco do handoff Gestor -> DP.

Análise/Avaliação:

5. Avalie as consequências de consultar muitas pessoas antes de uma decisão (C alto). Quais critérios você usaria para limitar quem deve ser consultado?
6. Você detectou um padrão de e-mails perdidos no DP. Como priorizaria ações (curto, médio e longo prazo) para mitigar esse problema?

Discussão (pontos para debate em sala):

A centralização de responsabilidades (um dono por caixa de e-mail) é sempre recomendável? Quais são vantagens e desvantagens?

Quando vale a pena transformar um atrito em um projeto de TI vs. mudar procedimento/treinamento?

Atividades Práticas sugeridas

Atividade 1: Mapeie o fluxo de um processo do seu time (3–6 etapas). Preencha R, A, C, I para cada atividade.

Atividade 2: Identifique todos os handoffs no seu fluxo e classifique o impacto caso cada handoff falhe.

Atividade 3: Para o handoff mais crítico, registre evidências (e-mails, número de ocorrências) e proponha ao menos duas soluções possíveis (uma de processo e uma técnica).

Atividade 4: Simule a transformação de um atrito em uma necessidade e desenhe o esboço de um PRD (problema, evidências, persona, proposta de valor).

Dicas para Superar Dificuldades

Comece simples: preencha primeiro o R e o A, depois adicione C e I conforme necessário.

Use evidências reais (e-mails, casos) para justificar riscos e priorizar soluções.

Não tente resolver tudo de uma vez: registre atritos e priorize por impacto e frequência.

Revise o mapeamento com os atores envolvidos para garantir aderência e identificar handoffs ocultos.

Se perceber múltiplos A necessários, divida a atividade em passos menores, cada um com seu A.

Conclusão

Mapear responsabilidades (RACE/RACI) e handoffs é uma prática essencial para reduzir atrasos, retrabalho e perda de informações em processos operacionais. Ao registrar atritos com evidências, você cria um caminho claro da dor (atrito) até a necessidade e, por fim, a solução — justificando investimentos e mudanças com base em fatos. Pratique o exercício mental de mapear processos e handoffs: é um treinamento de gestão que traz clareza e melhora a eficiência organizacional.

BLOCO 1 · AULA 06

Checklist e Guia Prático para Mapear o AS-IS de Processos

Objetivos de Aprendizagem

Definir o que é um mapeamento AS-IS e identificar suas partes essenciais.

Descrever a sequência e os artefatos necessários para construir um mapeamento de processo (escopo, mapa mental, fluxo, atritos, regras, RACI, handoffs).

Aplicar critérios de qualidade para revisar um mapeamento AS-IS (checklist).

Identificar erros comuns na construção do AS-IS e como evitá-los.

Transformar atritos identificados em necessidades (introdução para o próximo bloco).

Conceitos-chave

1. O que é AS-IS (mapa do estado atual)

Explicação: AS-IS é o mapeamento do processo como ele ocorre atualmente — não o ideal, mas o real. Documenta início, fim, atores, etapas, regras, pontos de atrito e transições.

Aplicação: Fundamental para diagnóstico antes de propor melhorias; base para o trabalho de consultoria, análise ou implementação.

2. Artefatos do mapeamento

Tela de início: identificação do processo (nome, notas).

Escopo: define o que está dentro e fora do processo, início e fim.

Mapa mental: esboço inicial com centros e ramos que descrevem atores e fluxos básicos.

Fluxo (flow): representação linear das etapas com verbos (ações) e responsáveis.

Folhas: últimas atividades/degraus do mapa mental que viram linhas no fluxo.

Atritos: dores, problemas ou "BOs" identificados em cada etapa (devem indicar causa, não apenas sintoma).

Regras de negócio: lógica testável que determina comportamentos ou decisões (sempre com condição lógica, preferivelmente que retorne sim/não).

RACI (ou RACE conforme referido): quem Responsabiliza (R), quem é Accountable (A — somente um A por atividade), Consultado (C) e Informado (I).

Handoff: passagem de responsabilidade entre atores — o "passar de bastão".

To-Be: o estado alvo/planejado (mapeado posteriormente) que receberá as regras e necessidades definidas.

3. Fluxo de trabalho sugerido (sequência prática)

1. Preencher a tela de início (nome do processo, notas).
2. Determinar escopo (início e fim claros; definir o que não entra).
3. Criar o mapa mental com os atores e principais passos.
4. Enviar mapa mental para o fluxo (cada "folha" vira uma atividade/linha).
5. Para cada atividade:
 - Definir verbo (uma ação por etapa).
 - Identificar responsáveis (RACI/RACE).
 - Registrar atritos com causa clara.
 - Registrar regras de negócio (testáveis).
 - Registrar handoffs quando houver mudança de responsável.
6. Revisar checklist de qualidade; corrigir itens faltantes.
7. Próximo passo: transformar atritos em necessidades (próxima aula/módulo).

Termos Técnicos

Termo	Definição	Contexto/Exemplo
AS-IS	Mapeamento do estado atual do processo	Documenta como o processo ocorre hoje, com atritos, regras e responsáveis.
Escopo	Limites do processo (início, fim e o que está fora)	Definir que o processo começa quando colaborador solicita e termina após assinatura do DP.
Mapa mental	Representação ramificada das etapas e atores	Centro: solicitação de férias; ramos: gestor aprova, DP processa, colaborador recebe.
Fluxo	Representação linear das etapas com verbos	Etapas: "Colaborador solicita", verbo: "Solicitar".
Folha	Última atividade de um ramo do mapa mental que vira atividade no fluxo	Cada atividade final do mapa mental vira uma linha no fluxo.
Atrito	Problema ou dor identificada numa etapa (com causa)	Ex.: "Gestor demora aprovação" — causa: falta de notificação.
Regra de negócio	Condição lógica que determina ação/decisão	"Se faltas > X, aprovar só com revisão" — deve ser testável.
RACI/RACE	Matriz de responsabilidades	R = Realiza, A = Aprova (somente 1), C = Consulta, I = Informa.
Handoff	Passagem de responsabilidade entre atores	Passar tarefas do gestor para o DP.
To-Be	Estado futuro desejado do processo	Resultado do projeto de melhoria, mapeado no módulo seguinte.

Orientações Detalhadas e Boas Práticas

Comece pelo real: mapear o que acontece hoje, não o que deveria acontecer.

Use verbos únicos por etapa: cada atividade deve ser descrita por um verbo claro (ex.: Solicitar, Aprovar, Enviar).

Aponte causas dos atritos: distinguir sintoma ("acho que demora") de evidência/causa ("demora X dias porque não há SLA").

Regras testáveis: para cada regra, descreva como testá-la (ex.: cálculo, validação booleana).

RACI correto: assegure que exista ao menos um R (realiza) e exatamente um A (aprovador) por atividade.

Handoffs explícitos: registre sempre onde o processo muda de dono; não deixe passagens implícitas.

Atualização do fluxo: se alterar o mapa mental, reenvie para atualizar o fluxo; o sistema incluirá apenas tarefas novas.

Documente o que está fora do escopo: evita escopo infinito e ambiguidades.

Erros Comuns e Como Evitá-los

Mapear o processo ideal em vez do real — Evite: faça observações, entrevistas e registros reais.

Misturar solução com diagnóstico — Evite: registre problemas e regras; deixe a solução para etapas posteriores.

Escopo infinito — Evite: delimite claramente o que está dentro/fora.

Regras vagas — Evite: descreva regras com lógica testável e condições específicas.

RACI sem dono — Evite: assegure responsável e aprovador claros.

Handoffs pulados — Evite: registre todas as transições de responsabilidade.

Checklist de Qualidade (Autoavaliação)

Marque e corrija itens faltantes antes de avançar. Se falharem dois ou mais, corrija antes de seguir.

- ☐ Nome do processo mapeado no fluxo.
- ☐ Início e fim do processo claramente definidos.
- ☐ Escopo determinado (inclui o que não entra).
- ☐ Mapa mental com centros e ramos completos.
- ☐ Fluxo preenchido com verbos únicos e responsáveis.
- ☐ Atritos registrados (com causa, não apenas sintoma).
- ☐ Regras documentadas e testáveis.
- ☐ RACI/RACE com pelo menos um R e exatamente um A por atividade.
- ☐ Handoffs identificados em todas as mudanças de responsável.
- ☐ Fluxo legível e coerente.
- ☐ Pelo menos 3 atritos mapeados.
- ☐ Pelo menos 5 regras registradas.
- ☐ Rastro/run-off do processo conferido.

Se não conseguir explicar o processo em 5 minutos para um colega, reveja o mapeamento.

Resumo dos Pontos Principais

O AS-IS é a fotografia do processo atual — foco no real.

A sequência prática: tela inicial → escopo → mapa mental → fluxo → detalhamento por atividade (verbos, RACI, atritos, regras, handoffs).

Atritos devem indicar causa; regras devem ser lógicas e testáveis.

Garantir um dono para cada etapa e registrar passagens de responsabilidade.

Evitar mapear soluções na etapa de diagnóstico.

O próximo passo (Bloco 2) é transformar atritos em necessidades — preparação para o To-Be.

Perguntas de Revisão e Prática

Compreensão

1. Explique em até 3 frases o que diferencia um mapeamento AS-IS de um To-Be.
2. Por que é importante que as regras de negócio sejam testáveis? Dê um exemplo.

Aplicação

3. Dê um exemplo de um atrito para o passo "gestor aprova" e proponha como identificar a causa verdadeira.
4. Em uma atividade com mais de uma pessoa envolvida, como você aplicaria o RACI? Dê um cenário e preencha R, A, C, I.

Análise/Discussão

5. Discuta um caso hipotético em que mapear o processo ideal (em vez do real) prejudicaria a solução final. Quais seriam as consequências?
6. Como você convenceria um gestor resistente a documentar atritos que aparentemente são "óbvios" e não merecem registro?

Desafio Prático

7. Pegue um processo simples da sua rotina (ex.: solicitação de material, aprovação de férias, compra de reposição) e:
- Desenhe rapidamente um mapa mental.
 - Liste 5 etapas do fluxo com verbo e responsável.
 - Identifique 2 atritos e descreva uma hipótese de causa para cada um.
 - Indique um handoff relevante.

Pré-requisitos e Conhecimentos Assumidos

Noções básicas de processos de negócio e terminologia (fluxo, stakeholders).

Familiaridade com conceitos de responsabilidade (papéis em equipe).

Habilidades de observação e entrevista para coletar dados reais do processo.

Dificuldades Comuns e Estratégias para Superá-las

Dificuldade em distinguir sintoma de causa: use dados (logs, tempos) e entrevistas para validar.

Escopo mal definido: comece pequeno; delimite início e fim com eventos concretos.

Resistência a documentar: mostre benefícios práticos (redução de retrabalho, clareza).

Confusão sobre RACI: treine com exemplos reais do negócio e padronize um modelo.

Próximos Passos

Revisar o checklist e ajustar o AS-IS até que esteja claro e explicável em 5 minutos.

Na próxima aula (Bloco 2): transformar atritos em necessidades, definir indicadores e preparar o To-Be.

BLOCO 2 · AULA 07

Mapeamento de Necessidades (Bloco 2) — Como transformar atritos em backlog priorizado

Objetivos de Aprendizagem

Definir o que é uma necessidade dentro do contexto de mapeamento de processos.
Analisar atritos mapeados para transformar problemas em necessidades mensuráveis.
Aplicar uma fórmula prática para redigir necessidades (persona → necessidade → benefício).
Priorizar necessidades usando critérios (impacto, esforço, viabilidade, must/should/could).
Avaliar quando usar Inteligência Artificial (IA) em uma necessidade.
Projetar indicadores e dashboards iniciais para acompanhamento de processos (ex.: férias).

Conceitos-chave

1. Ponte entre processo atual e solução: a Necessidade

Explicação: Após mapear como o processo funciona hoje (fluxo, responsabilidades, hand-offs, atritos), a necessidade descreve o que precisa ser melhorado para resolver aquele atrito.
Aplicação prática: Transformar um atrito identificado (ex.: "aprovação por e-mail se perde") em uma necessidade genérica e voltada ao resultado (ex.: "criar uma fila única de aprovações com prazo e status").

2. Não confundir necessidade com solução

Explicação: A necessidade descreve o resultado esperado para uma persona específica. A solução (tela, formulário, sistema, botão) vem depois.

Exemplo: Em vez de escrever "criar um agente A", escrever "gestor precisa saber o prazo final de cada solicitação para não perder o prazo de resposta".

3. Fórmula para escrever necessidades

Estrutura sugerida:

Como [persona], eu preciso de [capacidade/resultado] para [benefício esperado].

Personas comuns no exemplo: Colaborador, Gestor, Departamento Pessoal (DP).

Exemplo concreto: "Como colaborador, preciso saber quais são todos os campos que devo preencher para que minha solicitação de férias chegue completa ao gestor."

4. Uso do mapa mental + importação para a tabela de necessidades

Fluxo: Atritos mapeados → transformar cada atrito em um nó no mapa mental → cada nó vira uma necessidade na tabela.

Ferramenta: preencher texto da folha do mapa mental com a frase-formato; depois importar/mandar para a tabela de necessidades.

5. Critérios de priorização e avaliação

Campos importantes para cada necessidade:

Persona

Descrição da necessidade

Benefício (por que importa)

Impacto (por exemplo numa escala 1–5)

Esforço (complexidade de implementação; 1–5)

Viabilidade (dados, regras, recursos disponíveis)

Must / Should / Could (must = deve ser feito; should = deveria; could = opcional)

Uso de IA? (Sim/Não) — avaliar se a IA agrega valor ou é desnecessária

Exemplos de avaliação:

Necessidade simples e repetitiva → impacto alto, esforço baixo → must.

Necessidade que exige cruzamento complexo de dados históricos → pode requerer IA.

6. Quando usar IA

Indicadores para usar IA:

Necessidade de buscar, cruzar e sintetizar informações (ex.: "Quais foram as últimas compras?" ou "Previsão de vendas dos últimos dois anos").

Respostas que exigem interpretação, agregação ou geração de linguagem natural.

Quando não usar IA:

Regras lógicas simples (ex.: validar se a data foi preenchida). Nesse caso, automatização simples e validações são suficientes.

7. Exemplo prático: Processo de Férias — atritos transformados em necessidades

Atritos mapeados e necessidades geradas (resumo):

Colaborador esquece campos → Necessidade: saber todos os campos obrigatórios para enviar solicitação completa.

Colaborador não pré-avisa equipe → Necessidade: ser lembrado de informar pré-aprovação com a equipe.

Gestor demora >10 dias → Necessidade: gestor precisa saber o prazo final de cada solicitação (evitar perder prazo).

Colaborador quer status → Necessidade: visualizar quando enviou, status de aprovação e tempo decorrido.

Prazo mínimo para solicitação (15 dias) → Necessidade: informar ao colaborador o prazo mínimo no momento da solicitação.

Gestor não vê e-mail → Necessidade: notificar o gestor sobre nova solicitação (meio de notificação definido depois).

DP com múltiplos operadores → Necessidade: saber se um processo já começou a ser atendido por alguém do DP.

DP precisa de dashboard → Necessidade: dashboard com processos abertos, responsáveis, tempo em aberto e prazos próximos.

Gestor quer prova de leitura de negativa → Necessidade: registrar data em que a negativa foi lida pelo colaborador.

Terminologia Técnica

Termo	Definição	Contexto / Exemplo
Atrito	Problema ou ponto de dor identificado no fluxo atual	Ex.: "aprovação por e-mail se perde"
Necessidade	Descrição do que precisa melhorar, focada em resultado para uma persona	Ex.: "fila única de aprovações com prazo e status"
Persona	Papel/usuário que experiencia a necessidade	Ex.: Colaborador, Gestor, DP
Backlog priorizado	Lista de necessidades ordenadas por prioridade para implementação	Resultado do mapeamento e priorização
Must/Should/Could	Critério de prioridade: deve/ deveria/ poderia	Ajuda na definição do MVP
Viabilidade	Possibilidade técnica/organizacional de implementar	Verifica dados, regras, permissões
IA (Inteligência Artificial)	Uso de modelos para busca, interpretação ou geração de conteúdo	Usar quando há necessidade de agregação, previsão ou linguagem natural

Pontos de Resumo / Takeaways

O mapeamento de necessidades é a ponte entre entender "onde dói" e decidir "o que iremos construir".

Escrever necessidades deve focar na persona e no resultado, com tecnologia fora da primeira formulação.

A priorização deve considerar impacto, esforço, viabilidade e classificação must/should/could.

Nem toda necessidade precisa de IA — aplique IA apenas quando ela gerar valor claro (pesquisa, agregação, previsões, linguagem natural).

Mapas mentais ajudam a organizar atritos e transformá-los em necessidades estruturadas.

Preencher benefícios e estimativas (impacto/esforço) é essencial para decisões conscientes — não faça "nas coxas".

Possíveis Confusões e Como Evitá-las

Confundir necessidade com solução: sempre escrever a necessidade como resultado esperado, sem definir telas/técnicas.

Inclusão prematura de tecnologia: deixar ferramenta/IA/integração para a fase de solução/modelagem.

Ignorar o benefício: cada necessidade deve ter um benefício claro ligado ao problema que resolve.

Subestimar validações simples: muitas dores são resolvidas com regras simples, sem IA.

Perguntas de Revisão e Prática

Compreensão (nível fácil)

1. O que diferencia um atrito de uma necessidade?
2. Qual é a fórmula sugerida para escrever uma necessidade?
3. Por que não devemos incluir a tecnologia na redação inicial da necessidade?

Aplicação (nível médio)

4. Transforme o atrito "gestor não responde ao e-mail" em uma necessidade usando a fórmula *persona→necessidade→benefício*.
5. Dê três exemplos de quando a IA seria indicada para uma necessidade e três exemplos de quando não é necessária.

Análise/Discussão (nível avançado)

6. Você recebeu a necessidade "reduzir o tempo de resposta de aprovações". Proponha critérios (impacto, esforço, viabilidade) para priorizar essa demanda em relação a outras duas: "corrigir formulário com campos obrigatórios" e "criar dashboard para DP".
7. Debata os riscos de automatizar a notificação de leitura de mensagens (ex.: privacidade, confiança, integridade dos registros).

Exercício prático

8. Pegue um processo da sua organização (ou crie um cenário) e:
- Mapear 3 atritos.
 - Escrever 3 necessidades usando a fórmula.
 - Avaliar impacto, esforço e viabilidade e indicar must/should/could.
 - Indicar se a IA deve ou não ser usada, justificando.

Pré-requisitos / Conhecimentos Assumidos

Noções básicas de mapeamento de processos (fluxogramas, responsabilidades).
Compreensão de papéis organizacionais (colaborador, gestor, DP).
Conceitos básicos de priorização de tarefas e MVP.

Estratégias para Superar Dificuldades

Se estiver em dúvida sobre o benefício: pergunte "qual problema isso resolve?" e "para quem isso importa?".
Evite criar soluções prematuras; escreva a necessidade de forma genérica e teste com personas antes de definir tecnologia.
Use exemplos concretos (fluxo real) para validar impacto e esforço com stakeholders.
Faça backups regulares dos artefatos (JSON, mapas mentais) como demonstrado no tutorial.

BLOCO 2 • AULA 08

Entrevistas para Mapeamento e Validação de Processos

Objetivos de Aprendizagem

- Definir o propósito das entrevistas no mapeamento de processos.
- Identificar quem deve ser entrevistado com base no mapeamento prévio.
- Planejar e conduzir entrevistas sem enviesamento (curiosidade neutra).
- Registrar e classificar informações coletadas (fatos, dores, regras, pendências).
- Analisar respostas para ajustar o mapa de processo e registrar próximos passos.
- Avaliar evidências antes de transformar regras em necessidades ou soluções.

Conceitos Principais

1. Papel da Entrevista no Mapeamento de Processos

- A entrevista não é o ponto inicial do mapeamento; deve ser feita após um primeiro levantamento do processo.
- Objetivo: validar e aprofundar o mapa já elaborado, ouvindo quem vive o processo.
- Evita reiniciar tudo do zero; foca em confirmar e detalhar atritos, papéis e handoffs.

Aplicação prática:

- Após mapear fluxos e atividades, marque entrevistas com as pessoas já identificadas (executores, aprovadores, sustentadores, DP, gestores).

2. Seleção dos Entrevistados

Priorizar quem executa e sofre o atrito no dia a dia.

Incluir: colaboradores que solicitam, colaboradores que executam, gestores que validam, departamento pessoal (DP), quem sustenta o processo.

Não entrevistar apenas níveis hierárquicos altos (ex.: só diretor), porque eles tendem a validar, não a detalhar o dia a dia.

Exemplo:

Se o processo envolve solicitação de contratação: entrevistar quem solicita, quem aprova, quem consolida dados no DP e quem operacionaliza.

3. Preparação da Entrevista

Revisar mapa, fluxos, atritos e handoffs antes do encontro.

Definir objetivo claro: entender o processo atual e validar o que foi mapeado — não vender uma solução.

Escolher formato: preferencialmente presencial; remoto se necessário.

Agendar com duração adequada (30–45 minutos recomendados) e pedir consentimento para gravação (LGPD).

Preparar 6–8 perguntas focadas (melhor poucas perguntas bem feitas do que muitas superficiais).

Checklist pré-entrevista:

Objetivo definido

Mapa do processo revisado

Entrevistado selecionado

Roteiro com 6–8 perguntas

Horário combinado corretamente

Consentimento para gravação

Plano de registro (onde vai registrar fatos, regras, dores)

4. Roteiro de Perguntas (Estrutura Recomendada)

Abertura: contextualizar objetivo e tempo disponível.

Perguntas principais (exemplos):

1. Me conta o processo de início ao fim.
2. Onde hoje você está perdendo mais tempo?
3. Onde ocorrem mais erros ou retrabalho?
4. Quais exceções aparecem com frequência?
5. Que informação costuma faltar?
6. Como você costuma decidir quando existe dúvida?
7. O que aconteceria se o processo melhorasse 50%?
8. Se pudesse mudar uma coisa amanhã, qual seria?
 - Encerramento: resumo em 2–3 frases do que foi entendido, confirmação e próximos passos.

Dica:

Prefira perguntas abertas (Como? O que? Por que?) e peça exemplos concretos (ex.: "Da última vez que você fez isso, como foi?").

5. Postura do Entrevistador

Curiosidade neutra: demonstrar interesse sem induzir respostas.

Evitar promessas, debates ou sugestões de solução durante a entrevista.

Manter objetividade para não estender além do tempo.

Usar silêncio produtivo e parafrasear para confirmar entendimento:

Ex.: "Só pra ver se eu entendi: quando o gestor manda, você faz X, Y, Z. Faz sentido?"

Puxar a conversa de volta ao roteiro caso o entrevistado se desvie demais.

6. Registro e Classificação da Informação

Classificar cada observação como:

Fato: evento observável (ex.: "não respondeu fulano").

Regra: condição que determina um comportamento (ex.: "só posso enviar se tiver assinatura").

Dor/atrito: impacto que causa sofrimento, tempo perdido ou retrabalho.

Pendência: ações incompletas ou responsáveis não contatados.

Nem toda regra se torna um atrito; exigir evidência antes de transformá-la em necessidade.

Registrar consentimento para uso de citações ou para identificar a pessoa.

7. Uso da Gravação e Ferramentas

Gravar (com consentimento) para transcrever depois via IA ou manualmente.

Registrar notas durante a entrevista: anotar fatos, dores, regras e ideias.

Usar formulários ou sistema para armazenar entrevistas e imprimir quando necessário para uso offline.

8. Pós-entrevista: Próximos Passos

Revisar o mapa de processo com base nas novas informações.

Incluir novos atritos ou etapas que surgiram.

Validar se as dores identificadas foram adicionadas às necessidades.

Comunicar entrevistado sobre próximos passos e se será citado (anonimato/autoria).

Manter uma porta aberta para esclarecimentos futuros.

Terminologia Técnica

Termo	Definição	Contexto
Atrito / Dor	Problema que causa perda de tempo, erros ou insatisfação no processo	Ex.: etapas que geram retrabalho
Handoff	Transferência de responsabilidade entre papéis/ áreas	Ex.: de solicitante para aprovador
Regra	Condição formal que define como o processo deve ocorrer	Ex.: só enviar com aprovação assinada
Fato	Informação objetiva e verificável	Ex.: "a solicitação não chegou ao DP"
Pendência	Tarefa ou resposta ainda não concluída	Ex.: "esperando retorno do gestor X"
Curiosidade neutra	Postura de ouvir com interesse sem influenciar respostas	Evitar induzir soluções
LGPD	Lei Geral de Proteção de Dados; exige consentimento para gravação e uso de dados pessoais	Recolher consentimento para gravações/transcrições

Pontos de Resumo (Takeaways)

Entrevistas servem para validar e aprofundar um mapa de processos já mapeado, não para iniciar mapeamentos.

Escolher bem os entrevistados: priorizar quem vive o atrito.

Preparar e seguir um roteiro curto (6–8 perguntas) e objetivo.

Manter postura neutra e curiosa; evitar vender soluções.

Registrar fatos, regras e dores com evidências antes de transformar em necessidades.

Gravar quando possível (e autorizado) para garantir fidelidade das informações.

Encerramento claro com resumo e próximos passos é essencial para alinhamento.

Questões de Revisão e Discussão

Compreensão:

1. Por que a entrevista deve ocorrer após um primeiro mapeamento do processo?
2. Quais perfis de pessoas você deve priorizar para entrevistar? Justifique.

Aplicação:

3. Dado um processo fictício de aprovação de férias, liste três perguntas abertas que você faria e o que esperaria registrar como fato, regra ou dor.
4. Planeje uma agenda de 35 minutos para uma entrevista com um executor que normalmente tem pouco tempo. Como você abriria e encerraria a entrevista?

Análise e Avaliação:

5. Você encontrou uma regra declarada pelo entrevistado, mas sem evidência prática. Como você procede antes de considerar isso uma necessidade?
6. Discuta riscos de entrevistar apenas gestores e não os executores. Quais vieses podem surgir e como impactam o projeto?

Discussão em grupo:

7. Como você equilibraria curiosidade neutra e empatia quando o entrevistado demonstra frustração no processo?
8. Que medidas de conformidade com LGPD você implementaria em um projeto de mapeamento com múltiplas entrevistas?

Desafio prático:

9. Realize duas entrevistas curtas seguindo o roteiro de 8 perguntas. Registre fatos, regras e dores. Em seguida, atualize o mapa de processo com as novas evidências e apresente os próximos passos planejados.

Dificuldades Comuns e Estratégias para Superá-las

Entrevistado foge do assunto: redefina objetivos, use parafraseio e volte ao roteiro.

Entrevistado quer solução/debate: reforce escopo da entrevista (mapear, validar), anote a ideia como sugestão e marque para etapa futura.

Falta de tempo do entrevistado: priorize perguntas essenciais e peça exemplos concretos; agende follow-up se necessário.

Registro incompleto: grave com consentimento; se não for possível, peça permissão para enviar resumo por e-mail para confirmação.

Transformar regra em necessidade sem evidência: exija exemplos concretos ou dados (frequência, impacto) antes de priorizar.

Boas práticas finais:

Prepare-se: revise o mapa antes de entrevistar.

Seja breve e objetivo: 30–45 minutos é suficiente com foco.

Ouça com curiosidade neutra: registre evidências.

Resuma, combine próximos passos e mantenha comunicação clara com os envolvidos.

Boas entrevistas e bom mapeamento!

BLOCO 2 · AULA 09

Transformando Falas de Entrevista em Necessidades de Produto

Objetivos de Aprendizagem

Definir como transformar falas brutas de entrevistas em necessidades de produto (Definir).
Identificar atritos, dores e evidências a partir de entrevistas (Analisar).
Estruturar necessidade seguindo a fórmula "Como [persona], eu preciso [ação] para [benefício]" (Aplicar).
Avaliar a relevância de uma necessidade com base em frequência e impacto (Avaliar).
Priorizar e agrupar necessidades para facilitar a tomada de decisão (Organizar/Priorizar).

Conceitos Principais

1. Diferença entre Necessidade e Solução

Explicação: Necessidade descreve o resultado desejado a partir de uma dor ou atrito percebido pelo usuário. Solução descreve o que o sistema deve fazer para atender essa necessidade.

Exemplo: "Como colaborador, eu preciso acompanhar o estado da solicitação para não ficar no escuro" (necessidade). A solução seria: "O sistema deve mostrar o status da solicitação em tempo real" (funcionalidade — não é escrita agora).

Aplicação prática: Ao analisar entrevistas, registre apenas necessidades, não escreva imediatamente requisitos de sistema.

2. Origem das Necessidades: Atrito, Dor, Evidência

Explicação: A necessidade nasce de um atrito — uma situação que gera desconforto ou impede uma tarefa. Entrevistas alimentam fatos, regras e dores; a dor vira atributo e depois necessidade.

Exemplo: Falas como "Nunca sei se o gestor viu meu pedido" indicam um atrito (pedido desaparece no e-mail / fica sem status).

Fluxo: Fala bruta → identificar atrito/dor → registrar evidência/frequência → transformar em necessidade.

3. Fórmula da Necessidade

Estrutura: Como [persona], eu preciso [ação/resultado] para [benefício/porquê].

Exemplo completo: "Como colaborador, eu preciso acompanhar o estado da solicitação para não ficar no escuro nem cobrar o gestor via WhatsApp."

4. Validação: Frequência e Impacto

Explicação: Nem toda opinião vira necessidade. Antes de registrar, valide:

Frequência: Com que frequência ocorre? (sempre, muitas vezes, raramente)

Impacto: Quão severo é para o processo/usuário?

Diretrizes:

Se acontece raramente (ex.: uma vez ao ano), pode não ser prioridade.

Se não houver relação com nenhum atrito mapeado, marque como pendência e investigue mais.

Regra prática: "Sem dor identificada na entrevista → não vira linha nessa etapa."

5. Cuidado com Necessidades Fantasma

Definição: Necessidades baseadas apenas em opinião ou suposição sem evidências.

Abordagem: Registrar como pendência se falta evidência, ou coletar mais entrevistas/insumos.

6. Agrupamento e Priorização

Métodos de agrupamento:

Por etapa do processo (ex.: solicitação, aprovação, execução)

Por tipo (ex.: comunicação, acesso, visibilidade)

Por impacto (alto, médio, baixo)

Vantagem: Facilita priorização e ajuda a identificar padrões recorrentes entre entrevistas.

Terminologia Técnica

Termo	Definição	Contexto
Atrito	Situação que impede ou complica uma tarefa do usuário; causa da dor.	Ex.: "pedido some no e-mail" é um atrito que gera necessidade de rastreamento.
Dor	Problema percebido pelo usuário, consequência do atrito.	Ex.: "Nunca sei se o gestor viu meu pedido."
Evidência	Prova ou indício que confirma que a dor/atrito ocorre (fatos, frequência).	Ex.: múltiplas entrevistas relatando o mesmo problema.
Necessidade	Declaração de resultado desejado formulada sem descrever solução técnica.	Formato: "Como [persona], eu preciso [ação] para [benefício]".
Solução	Requisito ou funcionalidade que resolve a necessidade; descreve o que o sistema deve fazer.	Deve ser escrito mais adiante (fase to-be).
Necessidade Fantasma	Necessidade registrada sem evidências ou com baixo impacto/frequência.	Deve ser marcada como pendência ou descartada.

Sequência Lógica de Trabalho (Fluxo recomendado)

1. Conduzir entrevistas e anotar falas brutas.
2. Identificar potenciais atritos a partir das falas.
3. Registrar atritos no mapa/processo (se ainda não mapeados).
4. Validar frequência e impacto para cada atrito.
5. Transformar atritos comprovados em necessidades usando a fórmula (persona / ação / benefício).
6. Agrupar necessidades por etapa/tipo/impacto.
7. Priorizar para próxima fase (definição de soluções e requisitos técnicos).
8. Marcar pendências para investigar com mais entrevistas quando faltar evidência.

Exemplos Práticos

Exemplo 1:

Fala: "Nunca sei se o gestor viu meu pedido."

Atrito: Pedido some no e-mail; fica sem status.

Necessidade: "Como colaborador, eu preciso acompanhar o estado da solicitação para não ficar no escuro nem cobrar o gestor no WhatsApp."

Observação: Só registra se houver mais evidência; caso contrário, pendência.

Exemplo 2:

Fala: "Eu nunca sei qual é o saldo das minhas férias."

Perguntas de investigação: Onde está essa informação? Sempre preciso pedir ao DP? Qual é a frequência? Qual o impacto?

Possíveis resultados: Criar atrito novo (saldo inacessível) → necessidade: "Como colaborador, eu preciso consultar meu saldo de férias antes de solicitar" — validar com mais entrevistas.

Perguntas Comuns e Como Evitá-las (Dúvidas Frequentes)

"Devo escrever 'o sistema deve' ao registrar a necessidade?"

Não. Isso é solução. Mantenha foco no que o usuário precisa, não em como o sistema fará.

"Quantas entrevistas são suficientes para validar uma necessidade?"

Depende do contexto. Busque evidências repetidas e consistentes, priorizando frequência e impacto. Se for crítico, realize entrevistas complementares.

"O que fazer com relatos esporádicos?"

Marque como pendência e investigue mais; evite registrar como necessidade prioritária sem evidência.

"Uma fala pode gerar múltiplas necessidades?"

Sim. Uma fala pode derivar em mais de uma necessidade dependendo dos atritos identificados.

Pontos de Atenção e Possíveis Armadilhas

Transformar opinião isolada em necessidade sem validação.

Confundir necessidade com solução (escrever requisitos técnicos prematuramente).

Não mapear o atrito antes de criar a necessidade.

Falta de agrupamento que dificulta priorização posterior.

Atividades e Questões de Revisão

Compreensão

1. Defina com suas próprias palavras a diferença entre necessidade e solução.
2. Por que é importante validar frequência e impacto antes de registrar uma necessidade?

Aplicação

3. Dada a fala: "Moro longe e não consigo confirmar se meu atestado chegou ao RH", identifique o atrito, proponha as perguntas de investigação e escreva uma necessidade potencial.
4. Liste três formas de agrupar necessidades e explique quando usar cada uma.

Análise/Crítica

5. Analise um caso onde uma necessidade foi registrada com base em uma entrevista única. Quais passos você tomaria para validar ou refutar essa necessidade?
6. Discuta as consequências de pular a etapa de identificação de atritos e ir direto para especificar soluções.

Discussão em Grupo

7. Debata: "É melhor ter muitas necessidades bem documentadas ou poucas, bem validadas?" Justifique com exemplos de risco x benefício.
8. Imagine um processo em que várias pessoas precisam de informações do DP antes de agir. Como você estruturaria entrevistas para entender se é uma falha de processo ou de comunicação?

Nível Avançado

9. Projete um pequeno roteiro de entrevistas (5 perguntas) para identificar atritos relacionados ao processo de solicitação de férias.

- 10.** Com base em um conjunto de 10 entrevistas, descreva um critério objetivo para transformar um atrito em necessidade (por exemplo: ao menos 30% dos entrevistados reportaram o atrito e o impacto é médio ou alto).

Resumo Final

Transformar falas de entrevista em necessidades exige identificar atritos (a origem da dor), validar com evidências (frequência e impacto) e formular necessidades seguindo a estrutura "Como [persona], eu preciso [ação] para [benefício]".

Evite escrever soluções prematuramente; registre pendências quando faltar evidência.

Agrupamento e priorização são essenciais para transformar descoberta em roadmap de produto.

Pré-requisitos

Noções básicas de condução de entrevistas qualitativas.

Conhecimento do processo que está sendo mapeado (fluxo de trabalho ou jornada do usuário).

Capacidade de distinguir entre problema (o quê/porquê) e solução (como).

BLOCO 2 · AULA 10

Indicadores de Sucesso para Soluções e Produtos

Objetivos de Aprendizagem

Definir o que é um indicador de sucesso e reconhecer sua importância na validação de soluções (Conhecimento).

Identificar e selecionar indicadores relevantes que nascem de atritos ou necessidades (Compreensão).

Construir um indicador com baseline, fórmula, meta e prazo, incluindo fonte e frequência de coleta (Aplicação).

Avaliar o impacto de uma solução a partir da comparação de dados antes e depois da implementação (Avaliação).

Comunicar resultados de forma estratégica para carreira e organização (Análise/Criação).

Conceitos Principais

Indicadores de Sucesso (KPI)

Definição: Métricas que demonstram se a solução implementada alcançou o resultado esperado.

Importância: Transformam percepções em fatos mensuráveis; sustentam decisões, justificativas e comunicação de resultados (ex.: LinkedIn, portfólio).

Origem: Devem nascer de um atrito (dor) ou de uma necessidade mapeada durante entrevistas e análise do processo.

Atritos e Necessidades

Atrito: Problema operacional (ex.: dados incompletos, retrabalho, tempo excessivo).

Necessidade: Uma demanda que pode ser atendida por uma solução (ex.: melhorar notificações, reduzir custo).

Cada dor relevante merece ao menos uma forma de medição.

Baseline (Linha de Base)

Definição: Medida do estado atual do processo antes da implementação da solução.

Regra: Sempre medir o baseline (mesmo que aproximado). Sem baseline, não há como quantificar a melhoria.

Meta e Prazo

Meta: Valor desejado que indica sucesso (ex.: reduzir de 5 para 2 dias).

Prazo: Tempo para atingir a meta (ex.: 30, 60 dias). Torna o sucesso verificável.

Fórmula e Método de Cálculo

Cada indicador precisa de uma fórmula clara para cálculo.

Exemplos: porcentagem de dados incompletos = $(\text{n}^\circ \text{ de solicitações com dados faltantes} / \text{total de solicitações}) * 100$.

Fonte e Frequência de Coleta

Fonte: Sistema, planilha ou registro onde os dados serão extraídos.

Frequência: Periodicidade de medição (ex.: diária, semanal, a cada 30 dias).

Tipos de Indicadores e Exemplos Práticos

Tempo de Ciclo

O que mede: duração entre início e fim de um processo.

Exemplo: reduzir tempo médio de aprovação de 5 dias para 2 dias em 60 dias.

Taxa de Erro / Retrabalho

O que mede: frequência de reaproveitamento por dados incompletos ou incorretos.

Exemplo: reduzir a taxa de retrabalho por informação incompleta de 25% para 0% nos primeiros 5 dias após lançamento (quando se implementa bloqueio para envio incompleto).

SLA Cumprido (Porcentagem de Cumprimento de Prazos)

O que mede: proporção de solicitações atendidas dentro do prazo estipulado.

Exemplo: aumentar SLA de 80% para 99%.

Volume e Satisfação

Volume: número de solicitações processadas.

Satisfação: avaliação qualitativa por nota (0 a 10) de quem participa do processo.

Exemplo: aumentar satisfação de 2 para 6 em 30 dias.

Custo

O que mede: comparação de gastos antes e depois (ferramentas, horas homem, retrabalho).

Exemplo: reduzir custo mensal de ferramentas em X% após consolidação.

Como Construir um Indicador (Passo a Passo)

1. Identificar a dor/necessidade (através de entrevistas e mapeamento).
2. Selecionar o atrito relevante para medir.
3. Definir o nome do indicador.
4. Descrever a fórmula de cálculo (com clareza).
5. Registrar o baseline (valor atual).
6. Estabelecer a meta (valor alvo) e o prazo para alcançá-la.
7. Indicar a fonte dos dados e a frequência de coleta.
8. Planejar ações/soluções que suportam a melhoria (ex.: travar envio sem dados completos).
9. Documentar exceções e regras de negócio relacionadas (ex.: motivos de recusa).
10. Revisar periodicamente e ajustar se necessário.

Exemplo prático (caso de férias):

Indicador: Porcentagem de solicitações com dados incompletos.

Atrito: Colaborador envia solicitação sem data ou sem quantidade de dias.

Fórmula: $(\text{Número de solicitações recusadas por dados faltantes} / \text{total de solicitações}) * 100$.

Baseline: 25% (1 a cada 4 solicitações).

Meta: 0% nos primeiros 5 dias após implementação (sistema trava envio).

Fonte: Sistema de gestão de solicitações.

Frequência: verificar a cada 30 dias.

Outro exemplo (tempo de aprovação):

Indicador: Quantidade de dias para aprovar a solicitação.

Fórmula: data de aprovação/recusa - data de solicitação.

Métrica adicional: Contabilizar ocorrência quando >10 dias.

Baseline: 20% das solicitações ultrapassavam 10 dias.

Meta: reduzir para 2% em 60 dias (leva em consideração treinamento do gestor).

Terminologia Técnica

Termo	Definição	Contexto
KPI (Indicador-chave de Performance)	Métrica que indica sucesso de um objetivo	Usado para medir resultados-chave do projeto
Baseline	Valor inicial que representa a situação antes da solução	Necessário para comparação pós-implementação
SLA	Service Level Agreement: nível de serviço/prazo acordado	Medir % de solicitações atendidas dentro do prazo
Taxa de retrabalho	Proporção de processos que precisaram ser refeitos	Relacionado à qualidade e completude dos dados
Tempo de ciclo	Duração total de um processo do início ao fim	Usado para avaliar eficiência operacional
Fonte de dados	Sistema ou local onde os dados são registrados	Define confiabilidade e periodicidade da coleta

Pontos Resumidos (Resumo Executivo)

Sempre medir antes (baseline) e depois; sem baseline não há comparação objetiva.

Indicadores devem nascer de atritos/necessidades reais.

Defina nome, fórmula, baseline, meta, prazo, fonte e frequência para cada indicador.

Dois a três indicadores bem escolhidos valem mais que dezenas de métricas genéricas.

Combine indicadores quantitativos (tempo, % erros, custo) com qualitativos (satisfação).

Documente regras de negócio e motivos quando for necessário (ex.: motivo de recusa).

Use indicadores para comunicar impacto na carreira (portfólio/LinkedIn) e justificar investimentos.

Questões de Revisão e Prática

Compreensão:

1. Por que é essencial ter um baseline antes de implementar uma solução?
2. Qual a diferença entre um indicador bom e um indicador ruim? Dê exemplos.

Aplicação:

3. Para o processo de aprovação de viagens corporativas (atual demora média: 7 dias; motivo comum: falta de autorizações), proponha um indicador completo (nome, fórmula, baseline presumido, meta e prazo).
4. Imagine que 30% das solicitações de reembolso chegam com documentos faltantes. Proponha uma solução e um indicador para acompanhar a eficácia dessa solução.

Análise/Discussão:

5. Discuta vantagens e limitações de usar satisfação (nota 0-10) como indicador principal.
6. Como você priorizaria quais atritos merecem indicadores quando há muitos problemas identificados?

Desafio (nível avançado):

7. Projete um pequeno plano de implementação para dois indicadores (taxa de dados incompletos e tempo de aprovação), incluindo mecanismos automáticos de coleta, notificações e treinamento necessário aos gestores. Inclua como lidar com exceções e como validar a integridade dos dados.

Pré-requisitos e Conhecimentos Assumidos

Compreensão básica de processos de negócios e fluxo de trabalho.

Noções elementares de métricas e percentuais.

Capacidade de realizar entrevistas ou mapeamento de processos para identificar atritos.

Familiaridade com ferramentas básicas (planilhas, sistemas internos, dashboards) é desejável.

Áreas de Confusão Comuns e Como Superá-las

"Melhorar a experiência" sem métricas: transforme em metas quantificáveis (ex.: nota de 2 para 5).

Não medir baseline: volte às entrevistas e registros anteriores, solicite logs ou estime com cuidado, sempre documentando a incerteza.

Muitos indicadores: priorize por impacto e frequência; escolha 2–3 KPIs principais.

Confundir solução com indicador: indicador mede o problema/resultado; solução é a ação/técnica para atingir a meta.

BLOCO 2 · AULA 11

Priorização de Funcionalidades e Uso de IA em Projetos de Software

Objetivos de Aprendizagem

Definir critérios de priorização (impacto, esforço, viabilidade) e aplicar esses critérios para selecionar funcionalidades.

Explicar e aplicar o método MoSCoW (Must, Should, Could, Won't) para categorizar necessidades.

Analisar quando e como integrar Inteligência Artificial (IA) em uma aplicação, considerando riscos, custos e fallback.

Avaliar quais funcionalidades cabem em um MVP (Produto Mínimo Viável) com valor claro para o usuário.

Planejar estratégias de teste e validação (humana e automatizada) para recursos desenvolvidos com IA.

Conceitos-Chave

1. Prioridade: Por que priorizar?

Explicação: Nem tudo deve entrar na primeira versão de um produto. Priorizar evita sobrecarga de desenvolvimento e facilita entregas com valor rápido.

Contexto atual: A IA acelerou muito o desenvolvimento (tarefas que antes levavam semanas podem levar horas), mas ainda existe necessidade de priorização devido a testes, implementação e aceitação do usuário.

Aplicação prática: Selecionar o que entra no MVP com base em impacto real sobre o problema do usuário e esforço necessário.

2. Critérios de Avaliação: Impacto, Esforço e Viabilidade

Impacto (1–5): Quanto a funcionalidade resolve a dor principal do usuário? Quanto valor entrega?

Esforço (1–5): Quanto tempo/recursos é necessário para implementar e testar?

Viabilidade: Temos dados, regras e infraestrutura necessários para executar essa funcionalidade agora?

Como usar: Preencher esses três campos para cada necessidade registrada e então decidir a prioridade.

3. Matriz de Decisão e Quick Wins

Alto impacto + baixo esforço = Quick Wins (prioridade imediata).

Alto impacto + alto esforço = Projeto grande (priorizar conforme recursos e roadmap).

Baixo impacto + baixo esforço = Avaliar se vale a pena (pode piorar se poluir produto).

Baixo impacto + alto esforço = Evitar.

4. Método MoSCoW para Priorização

Must: Essencial para que o MVP entregue valor. Sem isso, o problema principal permanece.

Should: Importante, mas pode ficar para uma iteração futura sem quebrar o MVP.

Could: "Nice to have" — só se houver tempo/recursos sobrando.

Won't (ou Would/Won't have): Fora do escopo desta entrega; pode voltar em fases futuras.

Aplicação: Classificar cada necessidade com MoSCoW e registrar a justificativa (não apenas o rótulo).

5. MVP (Produto Mínimo Viável)

Definição: Primeira entrega que gera valor claro ao usuário e possui indicadores mensuráveis.

Como construir: Filtrar as necessidades marcadas como Must, validar que esses requisitos formam um conjunto coerente e mensurável.

6. Uso da IA: Onde faz sentido?

Bons candidatos:

Tarefas repetitivas de padrão textual (resumos, classificação de mensagens, extração de insights).

Classificação de histórico (ex.: pontuar probabilidade de compra).

Sugestões personalizadas.

Deteção de inconsistências, erros ou problemas (ex.: revisão de textos).

Maus candidatos:

Cálculos determinísticos simples (soma, PROC V, regras matemáticas) — resolvidos por scripts ou banco de dados.

Gravação de estado no banco (scripts/sistemas já resolvem isso).

Processos em que erro não é aceitável e não há fallback humano.

7. Riscos e Considerações ao Usar IA

Possíveis riscos: alucinações (respostas inventadas), vieses, privacidade e custos de tokens.

Erro aceitável?: Decidir se resultados aproximados são toleráveis ou se exigem supervisão humana.

Fallbacks:

Estratégias para quando a IA falhar (reprocessar com modelo melhor, usar um agente revisor, ou acionar humano).

Implementar mecanismos de validação automática (ex.: nota mínima; se abaixo, reprocessar ou enviar para humano).

Custos e modelos: Modelos mais avançados e precisos custam mais. Use modelo intermediário quando possível e eleve apenas quando necessário.

8. Testes e Implementação

Testes humanos: Essenciais para validação de usabilidade e aceitação.

Testes automáticos: Existem ferramentas (IA para testes) que ajudam, mas a revisão humana segue sendo importante.

Documentação: Gerar manuais e instruções para os usuários; IA pode ajudar a criar esses documentos, mas precisam ser revisados.

Terminologia Técnica

Termo	Definição	Contexto
MVP	Produto Mínimo Viável — versão inicial que entrega valor mensurável ao usuário	Use para definir o escopo da primeira entrega
MoSCoW	Técnica de priorização (Must/Should/Could/Won't)	Classificar necessidades e justificar escolhas
Impacto	Nível em que a funcionalidade resolve uma dor do usuário	Avaliação de 1 a 5
Esforço	Grau de complexidade e tempo para implementar	Avaliação de 1 a 5
Viabilidade	Se existem dados, regras e infraestrutura para executar agora	Sim/Não/Parcial
Token	Unidade de custo/uso em modelos de IA (pagar por processamento)	Relevante para estimar custos de IA
Alucinação	Resposta incorreta inventada pela IA	Risco que requer fallback/humana revisão
Fallback	Mecanismo alternativo caso a IA falhe	Pode ser outro agente de IA, um modelo superior ou humano

Resumo dos Pontos Principais

A IA reduz drasticamente tempo de desenvolvimento, mas não elimina a necessidade de priorização — testes, aceitação e infraestrutura ainda demandam tempo.

Priorize com base em impacto, esforço e viabilidade; use MoSCoW para categorizar necessidades.

Monte um MVP com funcionalidades Must que entreguem valor mensurável.

Use IA para tarefas repetitivas e com padrão textual, evitando usá-la para cálculos determinísticos ou ações que exigem garantia absoluta sem fallback.

Planeje fallbacks e revisões (humanas ou automatizadas) para mitigar riscos de alucinação, vieses e privacidade.

Documente justificativas ao marcar necessidade com IA (por que usar IA? qual o benefício?).

Questões de Revisão e Discussão

Compreensão

1. Defina em suas palavras os três critérios de priorização: impacto, esforço e viabilidade.
2. O que significa classificar uma necessidade como "Must" no método MoSCoW?

Aplicação

3. Dado um caso: sistema de gerenciamento de férias com notificações — classifique e justifique se o envio de e-mail deve ser Must, Should, Could ou Won't.
4. Cite três exemplos de funcionalidades apropriadas para uso de IA e três que não são.

Análise

5. Imagine que você integrou IA para resumir conversas com clientes. Descreva um plano de fallback caso a IA gere resumos incoerentes.
6. Avalie os trade-offs entre usar um modelo de IA mais barato vs. um modelo mais caro para a mesma tarefa.

Discussão (em grupo)

7. Em projetos corporativos, qual o papel do time de TI nas decisões sobre uso de APIs externas e IA? Quando o gestor deve "bater o pé" por uma necessidade ser Must?
8. Como você balancearia o desejo de incluir muitas funcionalidades (porque a IA facilita) com a necessidade de manter a experiência do usuário simples?

Desafios Práticos

9. Liste cinco funcionalidades do seu projeto atual e atribua impacto, esforço e viabilidade (1–5). Em seguida, classifique cada uma pelo MoSCoW e proponha o MVP.
10. Projete um pequeno fluxo onde a IA gera um texto (resumo), um segundo agente avalia a qualidade, e se abaixo do limiar, o humano revisa. Desenhe os passos e as responsabilidades.

Pré-requisitos e Conhecimentos Sugeridos

Noções básicas de gerenciamento de projetos ágeis (entregas, MVP).

Entendimento de conceitos de desenvolvimento de software (API, integração, banco de dados).

Noções sobre custos e funcionamento básico de modelos de IA/LLMs (tokens, modelos).

Familiaridade com planilhas (Excel), pois ajudam a diferenciar quando uma fórmula padrão resolve o problema.

Áreas de Confusão Comuns e Como Superá-las

"IA resolve tudo": Clarifique que IA é ferramenta para tarefas específicas (principalmente de inferência e linguagem), mas não substitui regras determinísticas simples.

"Se a IA falha, a culpa é do modelo": Nem sempre; falta de validação, ausência de fallback ou prompts mal desenhados podem ser raiz do problema.

"Mais funcionalidades = melhor": Muitas funções juntas podem sobrecarregar o usuário; priorize impacto sobre quantidade.

Como superar: Testes iterativos, validação com stakeholders e registro claro de justificativas e critérios.

Conclusão

Priorização é tanto técnica quanto humana: a IA tornou o desenvolvimento mais rápido, mas não eliminou a necessidade de escolhas racionais. Use impacto, esforço e viabilidade para filtrar o que entra no MVP; aplique MoSCoW para justificar cortes; e adote práticas de segurança e fallback ao integrar IA. Documente sempre as justificativas para que decisões sejam rastreáveis e discutidas com stakeholders.

BLOCO 3 · AULA 12

Como Cadastrar Soluções no PRD (To-Be) — Guia de Estudo

Objetivos de Aprendizagem

Definir o que é uma solução dentro do PRD To-Be e distinguir entre telas, funcionalidades e integrações.

Identificar o escopo de uma solução (o que entra e o que sai) e descrever o ciclo de status esperado.

Relacionar necessidades (must) mapeadas no AS-IS às soluções no To-Be.

Construir regras claras para cada solução e entender como vinculá-las.

Especificar os dados que o sistema deve guardar de forma prática e adequada ao uso de IA para geração do back-end.

Planejar a criação de soluções em ordem lógica para compor um PRD completo.

Conceitos-chave

1. PRD To-Be: Visão Geral

PRD (Product Requirements Document) To-Be descreve o produto que será construído com base nas necessidades levantadas (must) durante o mapeamento do AS-IS.

Não é um documento puramente técnico; é a história do que se espera que a IA (ou time de desenvolvimento) entregue.

2. Necessidade (Must) → Solução(s)

Cada atrito do AS-IS gera uma necessidade.

Uma necessidade classificada como "must" precisa ser atendida nesta entrega e pode gerar uma ou mais soluções.

Exemplos de tipos de solução: tela, funcionalidade, integração, dashboard.

3. Escopo da Solução

Definir claramente:

Objetivo do produto/solução.

O que está dentro do escopo (entrará no produto).

O que está fora do escopo (será tratado por outros sistemas ou não será incluído).

Ciclo de status (fluxo que os pedidos seguem — ex.: enviado ao gestor → aprovado → finalizado pelo DP).

4. Tipos de Solução

Tela: interface onde usuário interage (cadastro, visualização, confirmação).

Funcionalidade: processamento, cálculos, regras de negócio que não exigem interface.

Integração: conexão com sistemas externos (ex.: sistema de folha/DP).

Dashboard: visualização agregada para monitoramento.

5. Regras

Regras são instruções condicionais (if/then/else) explicadas claramente. Ex.: "Se o colaborador não falou com a equipe, não permitir envio".

Cada regra pode ser:

Mantida (usar como está do AS-IS),

Adaptada (reescrever para o To-Be),

Descartada.

Regras podem ser gerais do sistema ou atreladas a soluções específicas.

Regra bem descrita facilita que a IA implemente corretamente.

6. Dados que o Sistema Guarda (Banco de Dados / Back-end)

Indicar quais informações são necessárias (ex.: data de envio, solicitante, data de início de férias, quantidade de dias, se conversou com equipe).

Não detalhar colunas, tipos e índices prematuramente; permitir que a IA sugira e modele as tabelas com base nas necessidades.

Em casos complexos, orientar a IA com percepções de banco de dados só se houver experiência para tanto.

7. Ciclo de Cadastro e Validação (Exemplo: Solicitação de Férias)

Fluxo típico:

1. Colaborador faz login.
2. Colaborador preenche tela de solicitação (campos obrigatórios).
3. Confirmação pré-envio possível.
4. Status inicial: "Enviado para o gestor".
5. Gestor aprova ou recusa (se recusa, informa motivo).
6. DP finaliza e pede confirmação/assinatura do colaborador para concluir pagamento/processo.
 - Regras relacionadas: prazo mínimo de antecedência (ex.: 15 dias), motivo obrigatório em recusa, bloqueio se não conversou com equipe etc.

Organização do Conteúdo em Etapas Lógicas

1. Revisar AS-IS: identificar atritos e transformar em necessidades.
2. Priorizar necessidades como must/should/could.
3. Para cada necessidade must:
 - Criar uma ou mais soluções (tela, funcionalidade, integração).
 - Preencher: nome da solução, persona(s), objetivo, história (o que resolve), ação principal, próximo destino, dados que serão guardados, critérios de prontidão.
 - Vincular regras (manter/adaptar/descartar).
4. Repetir até mapear todas as necessidades críticas.
5. Validar o conjunto de soluções e regras — então gerar SDD (Software Design Document) e planejar fases de construção.

Exemplos Práticos (baseados no transcript)

Exemplo 1 — Tela: Cadastro de Solicitação de Férias (Colaborador)

Objetivo: Substituir e-mail por sistema de solicitação com todos os campos necessários.

Persona: Colaborador.

Campos sugeridos (conceituais): solicitante, data de solicitação, já conversou com a equipe (booleano), data de início pretendida, quantidade de dias.

Ação principal: Cadastrar e enviar para gestor.

Próximo destino: Tela de confirmação → envio → status "Enviado para gestor".

Regras associadas:

Bloquear envio se colaborador não confirmou ter conversado com a equipe.

Validar antecedência mínima de 15 dias; impedir envio se menor (ou exibir aviso/impedir conforme regra).

Critério de pronto: Usuário logado consegue enviar e visualizar status depois do envio.

Exemplo 2 — Funcionalidade: Calcular Tempo de Espera da Solicitação (Gestor)

Objetivo: Calcular e exibir tempo decorrido desde envio para priorização.

Persona: Gestor.

Onde aparece: Tela de solicitações com filtros e cores indicativas (alerta conforme aproximação do prazo).

Ação principal: Exibir lista ordenada da mais antiga para a mais nova, com destaque visual.

Regras/observações: Não guarda novos dados permanentes — cálculo baseado em timestamps.

Exemplo 3 — Dashboard: Lista de Solicitações para Aprovação (Gestor)

Objetivo: Permitir ao gestor visualizar rapidamente pendentes, aprovadas e recusadas.

Persona: Gestor.

Dados: resumo por status, contadores, filtros por data/persona, sinalização de prazos.

Ação principal: Visualizar e navegar para detalhamento/decisão.

Terminologia Técnica

Termo	Definição	Contexto de Uso
PRD To-Be	Documento que descreve o produto desejado após mudanças e melhorias	Usado para orientar construção da solução a partir das necessidades
AS-IS	Mapeamento do processo atual e seus atritos	Base para identificar necessidades
Necessidade (Must)	Requisito identificado como obrigatório para entrega	Gera soluções no To-Be
Solução	Elemento que resolve uma necessidade (tela, funcionalidade, integração)	Cada necessidade pode ter várias soluções
Regra	Condição de negócio descrita em formato if/then/else	Controla comportamento do sistema
Persona	Papel do usuário que interage com a solução (colaborador, gestor, DP)	Usada para definir objetivos e ações
Ciclo de status	Sequência de estados por que passa um pedido	Ex.: Enviado → Aprovado → Finalizado
Dashboard	Tela com visualização agregada de métricas/itens	Usada por gestores para monitoramento
SDD	Software Design Document — documentação técnica detalhada	Próxima etapa após PRD para execução técnica

Pontos de Resumo (Principais Aprendizados)

Transforme atritos do AS-IS em necessidades e depois em soluções no To-Be.

Cada solução precisa de objetivo, escopo (o que entra/ sai), ciclo de status e regras claras.

Prefira descrever dados necessários conceitualmente; não especifique colunas/tipos de BD prematuramente quando a IA for gerar o back-end.

Regras devem ser escritas de forma condicional e vinculadas a soluções; escolha manter, adaptar ou descartar regras originadas do AS-IS.

Uma necessidade pode gerar múltiplas soluções (tela + funcionalidade + integração).

Valide frequentemente: testes de prontidão devem indicar cenários concretos de uso.

Questões para Revisão e Prática

Compreensão

1. Defina a diferença entre uma necessidade (must) e uma solução no contexto do PRD To-Be.
2. Por que é recomendado não definir colunas e tipos de banco de dados de forma detalhada nesse estágio quando se usa IA?

Aplicação

3. Elabore uma regra condicional para uma tela de cadastro que impeça envio se o colaborador não tiver saldo de dias suficientes.
4. Proponha quais dados conceituais o sistema deve guardar para permitir que o gestor filtre solicitações por prioridade temporal.

Análise / Discussão

5. Discuta vantagens e riscos de permitir que a IA modele o esquema do banco de dados automaticamente.
6. Como você validaria que uma regra foi implementada corretamente por meio de critérios de pronto?

Desafio Prático

7. Para a necessidade "Gestor precisa ser notificado de novas solicitações", descreva:
 - Tipo de solução(s) necessárias.
 - Personas envolvidas.
 - Critério de prontidão (como saber que a solução está pronta).
 - Uma regra associada e sua exceção (se existir).

Possíveis Dúvidas / Áreas de Confusão

Confusão entre regra do AS-IS e regra do To-Be: sempre revisar se a regra antiga faz sentido no novo fluxo; usar manter/adaptar/descartar.

Quando detalhar o banco de dados: só em SDD ou se houver necessidade técnica explícita; durante PRD, indique apenas os dados conceituais.

Vinculação de regras: lembrar que regras podem ser gerais do sistema (ex.: fuso horário) ou específicas de uma solução (ex.: validar campo obrigatório).

Unidade de responsabilidade: diferenciar entre o que é tarefa de DP (fora do escopo do novo sistema) e o que será sinalizado/indicador pelo novo sistema.

Conclusão

Este guia orienta como transformar o mapeamento de processos e atritos (AS-IS) em soluções concretas no PRD To-Be, com foco em clareza de escopo, regras bem escritas e uso pragmático da IA para modelagem de dados e geração do back-end. A prática recomendada é iterar: descrever soluções, vincular regras, validar com usuários e só então avançar para a documentação técnica e construção em fases.

BLOCO 3 • AULA 13

Persistência de Dados e Arquitetura de Armazenamento para Soluções com IA

Objetivos de Aprendizagem

Definir os diferentes níveis de persistência de dados e distinguir entre origem e destino dos dados.
Analisar estratégias de armazenamento (navegador, arquivo, Superbase, Azure, Google, banco corporativo).

Aplicar conceitos de multi-usuário e multi-tenant ao desenho de soluções.

Avaliar requisitos de autenticação, provisionamento e migrações em projetos de software com IA.

Planejar a escolha de persistência para um MVP (produto mínimo viável) considerando políticas de empresa, sensibilidade de dados e integração com TI.

Conceitos Principais

1. Níveis de Dados: O que é armazenado vs. Onde é armazenado

Explicação: Existem duas formas de enxergar "dados":

Conteúdo do sistema: campos, colunas, relacionamentos (o que a IA geralmente gera automaticamente).

Persistência: onde esses dados "moram" (servidor local, nuvem, navegador, etc.).

Aplicação prática: Ao planejar um PRD (Product Requirements Document), especifique a persistência para evitar que a IA escolha por você (por exemplo, assumir PostgreSQL).

2. Estratégias de Persistência

Armazenamento somente no navegador:

Característica: dados locais ao navegador; útil para protótipos ou ferramentas simples.

Limitação: não é multi-usuário nem persistente entre dispositivos, salvo exportação/importação manual (CSV/JSON).

Arquivo (CSV, JSON, pasta na rede):

Característica: dados lidos a partir de arquivos; útil para integração simples com bases legadas.

Limitação: menos robusto para operações concorrentes e controle de acessos.

Banco na nuvem (Superbase, Azure, Google Cloud, etc.):

Superbase:

Vantagens: fornece autenticação, templates de e-mail (reset/confirm), APIs amigáveis para IA, multi-usuário pronto.

Customização: é possível (e muitas vezes desejável) substituir templates de e-mail por HTML com identidade da empresa.

Azure / Google:

Vantagens: integração com infraestrutura corporativa (ex.: Azure AD para login corporativo).

Observação: TI pode precisar provisionar o banco e conceder acessos.

Banco corporativo:

Característica: solução interna, frequentemente exigida por políticas de segurança/dados sensíveis.

Observação: requer coordenação com equipe de TI.

3. Origem vs. Destino dos Dados

Origem: de onde os dados entram no sistema (formularios, CRUD, importação de planilha).

Destino: onde os dados serão armazenados (Superbase, pasta de rede, banco interno).

Perguntas-chave: "Como os dados entram?" e "Para onde serão levados?"

4. Multi-usuário vs. Multi-tenant

Multi-usuário:

Definição: vários usuários (colaboradores, gestores, RH) acessando e com permissões diferentes dentro de uma única organização.

Uso comum: sistemas internos de empresa.

Multi-tenant:

Definição: diversas empresas (tenants) compartilham a mesma aplicação, com dados isolados por empresa.

Implementação: normalmente usa um único banco com separação por IDs/tenant_id (não um banco por cliente).

Vantagem: escalabilidade e facilidade de gestão.

Consideração comercial: se a solução será vendida como SaaS, projetar para multi-tenant.

5. Autenticação, Provisionamento e Primeiros Usuários

Tela de login vs. acesso aberto:

Decisão de projeto: incluir tela de cadastro pública, apenas por convite, ou integração com login corporativo.

Primeiro usuário (seed):

Seed: script que cria usuários iniciais (por exemplo, um admin de RH).

Alternativas: primeiro cadastrado vira admin; criação por convite; integração com AD.

E-mails de sistema:

Reset de senha, confirmação de conta: Superbase fornece templates padrão; decidir se usa-os ou customiza-los com identidade da empresa.

6. Migrações (Migrations)

Definição: scripts que documentam e aplicam mudanças no esquema do banco (criação/alteração de tabelas/colunas).

Importância:

Mantém histórico das alterações.

Permite reproduzir ou recriar o banco a partir de documentação.

Evita alterações "no olho" diretamente no banco sem registro.

Formato comum: SQL ou arquivos de migration gerenciados por frameworks.

Prática recomendada: nunca alterar o banco sem passar por uma migration.

7. Perfis de Profissionais Relevantes

Citizen Developer:

Definição: profissional de negócio (RH, finanças, marketing) que cria aplicações usando plataformas visuais ou low-code/no-code, sem formação profunda em programação.

HIC (High Impact Individual Contributor):

Definição: profissional sênior de alto impacto que, com automação e IA, produz trabalho equivalente ao de equipes inteiras.

Relevância: descreve o público-alvo/perfil que essa disciplina pretende formar.

Terminologia Técnica

Termo	Definição	Contexto de uso
Persistência	Local onde os dados são armazenados de forma duradoura	Escolha entre navegador, arquivo, Superbase, Azure etc.
Seed	Script ou operação inicial que insere dados padrão (ex.: usuário admin)	Usado para criar o primeiro usuário/configuração do sistema
Migration	Arquivo/script que documenta mudanças no esquema do banco	Garante histórico e reprodutibilidade das alterações
CRUD	Conjunto básico de operações: Create, Read, Update, Delete	Como os usuários irão inserir e manipular dados
Multi-usuário	Vários usuários dentro da mesma organização	Controle de permissões e diferentes telas/visões
Multi-tenant	Várias organizações (tenants) isoladas na mesma aplicação	Modelo SaaS, precisa isolar dados por tenant_id
Superbase	Plataforma de backend pronta com autenticação, APIs e templates	Muito usada para MVPs e integração com IA
PRD	Product Requirements Document	Documento que define requisitos do produto (inclui persistência)
AD / Azure AD	Active Directory da Microsoft	Integração para login corporativo

Pontos Resumidos / Principais Mensagens

Especifique a persistência de dados no PRD; sem isso a IA assumirá um padrão.

Há diferenças claras entre “o que o sistema guarda” (modelo de dados) e “onde os dados moram” (persistência).

Escolha entre armazenamento local (navegador), arquivo, banco na nuvem ou banco corporativo conforme requisitos e governança.

Multi-usuário é comum em aplicações internas; multi-tenant é necessário para SaaS que atenderá várias empresas.

Defina como serão gerados os primeiros usuários (seed) e como será o fluxo de cadastro/login. Pense em e-mails de sistema.

Migrações são essenciais para rastreabilidade das mudanças no banco e devem ser parte do fluxo de desenvolvimento.

Coordene com TI quando for usar infra corporativa (Azure/AD, banco interno).

Profissionais Citizen Developer e HIC são papéis emergentes que tornam possível criar soluções sem equipes extensas.

Questões para Reflexão e Discussão

Com base no seu caso de uso, qual persistência faz mais sentido: Superbase, Azure, Google, arquivo ou navegador? Justifique com três motivos.

O seu sistema precisará ser multi-tenant no futuro? Quais mudanças arquiteturais isso exige desde o início?

Quais políticas da empresa poderiam impedir o uso de soluções externas (ex.: Superbase)? Como você mitigaria esse risco?

Como você implementaria o processo de provisionamento do ambiente em cooperação com a equipe de TI?

Que dados do seu projeto são sensíveis e exigiriam controles adicionais (criptografia, acesso restrito, logs de auditoria)?

Prefere usar os templates de e-mail padrão do Superbase ou personalizá-los? Liste prós e contras.

Exercícios Práticos

1. Cenário (nível básico): Você precisa montar um protótipo de formulário de solicitação de férias para testes internos rápidos. Escolha a persistência e justifique: navegador, arquivo CSV/JSON ou Superbase? Quais limitações você prevê?
2. Cenário (nível intermediário): Seu departamento quer uma solução para gerenciamento de férias que várias filiais usarão, mas não para outras empresas. Defina se será multi-tenant ou single-tenant e descreva o modelo de autenticação recomendado.

3. Cenário (nível avançado): Imagine que o produto vá se tornar um SaaS. Esboce a estratégia de migrações, separação de dados por tenant e processos de criação do primeiro usuário (seed), incluindo considerações de segurança e escalabilidade.
4. Tarefa prática: Crie um roteiro (em 5 passos) para provisionar um ambiente Supabase e configurar um seed admin e os templates de e-mail com identidade da empresa.

Possíveis Dúvidas e Equívocos Comuns

"A IA cuida de tudo, não preciso decidir persistência." — Equívoco: sem instrução, a IA assumirá padrões que podem não atender requisitos legais, de segurança ou de negócio. Defina explicitamente a persistência no PRD.

"Multi-tenant exige um banco por cliente." — Equívoco: normalmente usa-se um único banco com separação lógica (tenant_id); bancos separados por cliente são custosos e difíceis de escalar.

"Migrações são opcionais." — Equívoco: sem migrations, você perde histórico e reprodutibilidade das mudanças no esquema do banco.

"Supabase já resolve todos os problemas." — Observação: Supabase agiliza muito, mas políticas corporativas, integrações AD, e customizações (ex.: e-mails com marca) ainda exigem decisões e trabalho.

Prerrequisitos e Conhecimentos Assumidos

Noções básicas de modelos de dados (tabelas, colunas, chaves).

Entendimento de CRUD e operações básicas de banco de dados.

Conhecimento elementar sobre arquiteturas em nuvem (Azure, Google Cloud) e serviços de backend.

Conceitos básicos de autenticação (login, reset de senha, convite).

Estratégias para Superar Desafios Técnicos

Ao trabalhar com TI para provisionamento, documente requisitos claros (endpoints, credenciais, permissões) e ofereça um plano de testes.

Para dados sensíveis, implemente controle de acesso baseado em funções, criptografia em trânsito e em repouso, e logging/auditoria.

Use migrations desde o início para evitar "drift" entre desenvolvedores e ambientes.

Se precisar escalar para multi-tenant no futuro, projete o schema já considerando tenant_id nas tabelas-chave para facilitar a evolução.

Resumo Final

Planejar a persistência de dados é uma decisão estratégica que influencia arquitetura, segurança, experiência do usuário e escalabilidade do produto. Defina claramente a origem e destino dos dados, escolha entre soluções rápidas (navegador/arquivo) ou robustas (Superbase/Azure), decida sobre multi-usuário vs. multi-tenant, determine fluxos de autenticação e primeiro usuário (seed), e use migrations para manter histórico e reprodutibilidade. Profissionais Citizen Developers e HIC estão no centro dessa nova era, capazes de criar soluções de alto impacto com o suporte de IA e plataformas modernas.

BLOCO 3 • AULA 14

Documento de Estudo: Construção de um PRD (Product Requirements Document) e Spec-Driven Development (SDD)

Objetivos de Aprendizagem

Definir o que é um PRD e explicar sua importância na construção de produtos com IA.

Analisar a estrutura típica de um PRD (blocos e sessões) e identificar quais informações devem ser preenchidas.

Aplicar o conceito de Spec-Driven Development (SDD) para transformar um PRD em fases executáveis.

Planejar a divisão de um projeto em fases (Expect) e organizar essas fases como arquivos Markdown persistentes.

Avaliar qualidade do PRD usando checklists, badges de cobertura e critérios de aceite.

Selecionar o modo de construção adequado (HTML único, framework/Aplicativo com Nuxt.js, ou início do zero) com base nas necessidades do produto.

Conceitos-chave

PRD (Product Requirements Document)

Definição: Documento que especifica o que será construído, por que e para quem. Serve como "fonte da verdade" para a IA e para a equipe de desenvolvimento.

Função: Base para que a IA (ou desenvolvedores) gerem o sistema, reduzindo retrabalho, alucinações e consumo excessivo de tokens.

Estrutura sugerida: dividido em 4 blocos com 16 seções (ex.: cabeçalho, autor, modos de construção, intenção, diagnóstico, objetivos, personas, necessidades, soluções, dados/persistência).

Boas práticas: preencher todas as seções; ligar necessidades (must) a soluções; definir "pronto quando" (critério de aceite).

Spec-Driven Development (SDD) / Expect-Driven Development

Definição: Método de transformar um PRD amplo em um conjunto de specs (expectations) menores e executáveis em fases.

Motivo: PRDs grandes demais dificultam a execução direta por IA, que pode se perder se solicitada a implementar tudo de uma vez.

Procedimento:

1. Gerar PRD completo (contexto e regras).
2. Solicitar à IA que "separe em fases".
3. Salvar cada fase como arquivo Markdown (fase_1.md, fase_2.md, ...).
4. Planejar execução da fase (modo plan) para gerar tasks e implementar incrementalmente.
 - Vantagens: persistência do contexto (arquivos), menor risco de alucinação, possibilidade de transição entre ambientes (ex.: Cursor → VS Code).

Modularização por Fases e Arquivos Markdown

Cada fase corresponde a um escopo menor do PRD que pode ser implementado isoladamente.

Salvar cada fase em arquivo facilita continuidade do trabalho e evita perda de contexto ao trocar ambientes.

Exemplo de fluxo: PRD grande → pedir divisão em fases → gerar markdowns por fase → pedir à IA para "planejar" a fase 1 → executar tasks.

Modos de Construção (Escolha do tipo de entrega)

HTML único (dashboard): arquivo HTML único para visualização fácil e distribuição.

Software/Aplicativo baseado em framework: aplicação com servidor, banco de dados, telas e workflows (ex.: Nuxt.js + Tailwind).

Começar do zero: sem template, tudo gerado do início.

Template padrão (padrão da pós): pacote template (zip) otimizado para acelerar desenvolvimento (já traz menu, tabelas, dark mode, etc.).

Escolha depende da complexidade da solução, necessidades de persistência, interação e requisitos de integração.

Persistência de Dados e Modelagem (Banco, Colunas, Tabelas)

A IA pode inferir colunas e tabelas a partir das regras e fluxos descritos no PRD.

Importante indicar se haverá persistência (banco de dados) e que tipo de armazenamento será usado.

Regras mapeadas no Exis (como é hoje) servem de contexto, mas nem todas precisam ir direto para o sistema — devem ser convertidas em regras do Tubi (solução desejada).

Checklists, Badges e Critérios de Aceite

Badges evidenciam o estado do PRD: se existem pendências, número de necessidades, se as necessidades têm soluções vinculadas, entre outros.

Checklist ajuda a validar se o PRD está completo: início do processo, regras, responsáveis, handoffs, necessidades, KPIs, uso de IA, telas, integrações, critérios de aceite.

Critério de aceite ("pronto quando"): indicador por solução que descreve as condições para considerar a implementação concluída.

Terminologia Técnica

Termo	Definição	Contexto
PRD	Product Requirements Document: documento de requisitos do produto	Base para desenvolvimento por IA/equipe
SDD / Spec-Driven Development	Metodologia que transforma PRD em specs (expectations) por fases	Permite execução incremental e reduz erros
Expect	Unidade de especificação/fase dentro do SDD	Cada Expect vira um arquivo markdown executável
Markdown	Formato de texto leve para documentação	Usado para armazenar cada fase de forma persistente
Template padrão	Pacote pré-configurado de arquivos e estrutura (Nuxt, Tailwind)	Facilita a geração de aplicações com padrões prontos
Must	Necessidade essencial (prioridade alta)	Deve estar vinculada a solução para evitar PRD ruim
Critério de aceite	Condição que define "pronto" para uma solução	Evita mal-entendidos sobre entregáveis
Alucinação (IA)	Resposta incorreta ou inventada pela IA	Reduzida com PRD detalhado e fases persistidas

Estrutura sugerida do PRD (Resumo das seções essenciais)

1. Cabeçalho: autor, data, modos de construção, escopo do documento.
2. Diagnóstico (como é hoje / Exis): processos atuais, atritos, dados de entrada/saída.
3. Objetivo do produto: o que se deseja alcançar.
4. Personas / Stakeholders: quem usa, quem decide, responsáveis.
5. Necessidades (Must / Moscow): lista priorizada (idealmente 1–5 musts para MVP).
6. Soluções propostas (Tubi): vínculo entre necessidades e soluções, telas e funcionalidades.
7. Dados e Persistência: onde os dados são armazenados, tipo de banco, modelos.
8. Regras de negócio: validações, fluxos, automações.
9. Critérios de aceite: "pronto quando" para cada solução.
10. Checklists e Badges: indicadores de completude do documento.
11. Anexos: arquivos templates, prompts de fase, prompts de execução.

Guia passo a passo (Fluxo prático para montar e usar o PRD)

1. Preencha o planejador/escopo no sistema (contexto, riscos, necessidades, soluções, telas).
2. Gere o PRD (um clique no app do curso).
3. Revise o PRD: verifique se todas as seções estão preenchidas e se cada necessidade tem solução.
4. Solicite à IA: "Separe em fases" — ela deve propor as phases/Expect.
5. Salve cada fase como arquivo Markdown (fase_1.md, fase_2.md, ...).
6. Ative "modo plan" (ou equivalente) para a fase que irá implementar; a IA gera tasks e um plano de execução.
7. Execute as tasks incrementalmente (evita pedir para IA construir tudo de uma vez).
8. Use checklists e badges para validar cobertura; corrija pendências (solutions sem vínculo, critérios de aceite faltando).
9. Quando a fase estiver aprovada (critérios de aceite cumpridos), passe para a próxima fase.

Exemplos e Aplicações Reais

Exemplo 1: Dashboard corporativo simples

Escolha: HTML único ou template padrão.

PRD enxuto: 3–5 necessidades, telas básicas, sem backend complexo.

Execução: possivelmente construir tudo de uma vez se PRD for pequeno.

Exemplo 2: Aplicativo empresarial com autenticação, roles e banco

Escolha: framework (Nuxt.js) + template padrão.

PRD completo: mapeamento de regras, persistência, integrações.

Execução: dividir em fases (login, CRUD de entidades, integrações, dashboards).

Exemplo 3: Projeto grande (múltiplos fluxos e integrações)

Procedimento: PRD amplo → gerar 6–8 fases → salvar cada fase em markdown → implementar em sprints.

Possíveis Áreas de Confusão e Como Evitá-las

"Pedir pra IA fazer o PRD sem insumos": gera PRD genérico e fraco. Evite. Forneça contexto, atritos, necessidades e soluções.

"Executar PRD inteiro de uma vez": IA pode se perder; divida em fases.

"Não vincular necessidades a soluções (must sem solução)": causa badges de pendência. Sempre vincular.

"Falta de critérios de aceite": dificulta perceber se uma solução está realmente pronta. Defina sempre o "pronto quando".

"Guardar apenas em memória da IA": perda de contexto ao mudar ambiente. Salve fases em arquivos persistentes (markdown).

Checklist Prático para Avaliação de Qualidade do PRD

- ☐ Todas as 4 seções principais preenchidas (diagnóstico, objetivo, personas, dados).
- ☐ Cada necessidade (must) tem solução vinculada.
- ☐ Critérios de aceite definidos para todas as soluções.
- ☐ Persistência de dados definida (sim/não, tipo de banco).
- ☐ Escolha de modo de construção definida (HTML / Framework / Zero).
- ☐ Template padrão identificado (se aplicável).
- ☐ Number de needs do MVP ajustado (ideal 1–5 musts).
- ☐ Arquivos de fase gerados e salvos em Markdown.
- ☐ Plano de execução (modo plan) para a fase atual.

Pontos Críticos e Recomendações

Não comece projetos sérios sem um PRD robusto: evita soluções genéricas e retrabalho.

Prefira dividir grandes PRDs em fases: reduz alucinação, melhora rastreabilidade.

Use templates (quando aplicável) para acelerar desenvolvimento e manter padrões.

Sempre definir "pronto quando" para cada entrega pequena.

Mantenha documentação persistente (markdowns), não dependa só da memória da IA.

Questões de Revisão (para estudo ou discussão)

Compreensão

1. O que é um PRD e por que é essencial antes de iniciar um projeto com IA?
2. Quais são as quatro seções principais que compõem um PRD segundo a aula?

Aplicação

3. Dê um exemplo de como você dividiria um PRD para um aplicativo de cadastro de clientes em três fases.
4. Explique como o uso de um template padrão (Nuxt.js) altera as instruções que você dará à IA.

Análise

5. Quais são as principais vantagens do Spec-Driven Development em comparação com pedir à IA para construir tudo de uma vez?
6. Analise um cenário em que salvar as fases apenas na memória da IA pode prejudicar o projeto.

Avaliação

7. Avalie a qualidade do seguinte item do PRD: "Necessidade: Relatório de vendas. Solução: gerar relatório." O que falta para tornar isso aceitável?
8. Proponha um checklist mínimo para garantir que uma fase esteja pronta para implementação.

Criação

9. Escreva um prompt (em alto nível) para pedir à IA que separe um PRD grande em fases.
10. Modele um critério de aceite para uma função de upload de imagem em um perfil de usuário.

Respostas sugeridas (breves)

1. PRD é o documento que descreve o que será construído, por que e para quem; é essencial para orientar a IA e reduzir erros/alucinações.

2. Diagnóstico (Exis), Objetivo do produto, Pessoas/Needs, Dados/Persistência.
3. Exemplo: Fase 1 — Autenticação e cadastro; Fase 2 — CRUD de clientes; Fase 3 — Relatórios e dashboards.
4. O template já traz componentes e padrões (menu, responsividade, estilos), logo a IA não precisa reinventar estrutura básica.
5. Permite execução incremental, persistência de fases em arquivos e redução de alucinação.
6. Perde-se contexto ao trocar de ambiente; longo tempo entre interações pode levar a perda de informações importantes.
7. Falta detalhar formato do relatório, fontes de dados, filtros, periodicidade e critérios de aceite (pronto quando).
8. Exemplo: documentado, tarefas criadas, critérios de aceite definidos, testes automatizados/aceitação manual, dependências resolvidas.
9. Prompt alto nível: "Pegue este PRD completo, avalie complexidade e separe-o em N fases priorizadas, listando para cada fase objetivos, entregáveis e dependências."
10. Critério de aceite exemplo: "Upload aceito quando o usuário conseguir enviar imagem <=5MB, formato PNG/JPG; pré-visualização exibida; imagem armazenada e URL retornada; testes unitários cobrindo validação."

Resumo Final

Um PRD bem feito é essencial para que a IA gere soluções úteis e menos genéricas.

Dividir grandes PRDs em fases (SDD) e persistir essas fases como arquivos Markdown aumenta a confiabilidade e permite continuidade entre plataformas.

Use templates e escolha o modo de construção apropriado conforme a complexidade do projeto.

Valide o PRD com checklists, badges e critérios de aceite antes de pedir à IA para construir qualquer fase.

BLOCO 3 • AULA 15

PRD e Kit de Construção: Guia Prático para Spec Driven Development (SDD)

Objetivos de Aprendizagem

Definir o que é um PRD (Product Requirements Document) e qual seu papel no desenvolvimento de produto.

Explicar o conceito de Kit de Construção e sua relação com Spec Driven Development (SDD).

Demonstrar como gerar, organizar e atualizar o PRD e o kit (árvore de pastas, arquivo agents, specs).

Aplicar um fluxo de trabalho baseado em fatias (fases/specs) para delegar tarefas à IA de forma controlada.

Avaliar riscos, critérios de aceite e quando marcar uma fase como "pronta".

Conceitos-chave

PRD (Documento de Requisitos do Produto)

Descrição: Documento que contém o "o quê" e o "porquê" do produto — necessidades, soluções, regras, persistência, critérios de aceite e contexto (as-is / to-be).

Função: Servir como fonte de verdade para construir o produto; registrar decisões e escolhas (por exemplo: estratégia de persistência, templates e stack).

Observação: Deve existir um PRD por produto. Não inclua pedidos de implementação diretos no PRD; esse arquivo é referência.

Kit de Construção

Descrição: Pacote gerado a partir do PRD que contém artefatos para orientar a implementação automatizada por IA.

Componentes principais:

Árvore de pastas (estrutura onde documentos e códigos devem residir)

Arquivo `agents` (regras/instruções para o agente de IA)

Specs (arquivos `.md` representando as "fases" do projeto)

Prompts sequenciais prontos para execução pelo agente

Função: Padronizar onde e como o agente buscará informações e executará tarefas.

Spec Driven Development (SDD) / "Fatias"

Descrição: Método de dividir o PRD em fatias de trabalho (fases/specs) que serão planejadas e executadas sequencialmente.

Fluxo resumido:

1. Gerar PRD.
2. Pedir à IA para propor as fases (índice).
3. Gerar a spec da Fase 1 (arquivo MD) com necessidades, regras e critérios "pronto quando".
4. Planejar a execução da spec (modo "plan").
5. Implementar apenas a fase aprovada.
6. Validar critérios e marcar como concluída.
7. Repetir para fases seguintes.
 - Vantagem: Evita sobrecarga de contexto (mitiga alucinações da IA) e traz controle e rastreabilidade.

Estrutura recomendada do fluxo de trabalho (sequência lógica)

1. Criar e revisar o PRD (to-be como fonte de verdade; incluir persistência, regras, critérios de aceite, riscos).
2. Exportar kit de construção (gerar árvore de pastas, arquivo `agents`, specs e prompts).
3. Guardar PRD junto ao kit na pasta `docs` do projeto.
4. Solicitar à IA a proposta de fases (arquivo índice/checklist).
5. Validar e aprovar o recorte das fases.

6. Gerar spec de cada fase individualmente (arquivo .md em docs/specs/fase-x.md).
7. Colocar agente em modo de planejamento para a fase aprovada.
8. Revisar plano, aprovar e pedir execução (implementar somente a spec aprovada).
9. Conferir critérios "pronto quando" e marcar fase como concluída em arquivo docs/fases.
10. Atualizar PRD e regenerar kit caso haja mudanças de necessidade/arquitetura/stack.

Relações entre conceitos

PRD → Kit de Construção: o PRD é a origem; o kit é a tradução do PRD para instruções executáveis por agentes.

Kit (agents + specs + prompts) → IA: padroniza entendimento e comportamento esperado do agente.

Specs → Plano de Implementação: cada spec é um contrato de implementação com critérios de aceite claros.

Marcação de fase concluída → Controle de fluxo: evita reimplementação desnecessária e mantém histórico.

Exemplos e Aplicações Práticas

Exemplo prático (resumido):

Produto: Sistema de solicitação de férias (substitui processo por e-mail).

PRD inclui:

Contexto (as-is): processos atuais por e-mail (não recriar).

Necessidades (MUST): campos obrigatórios, etapas, personas (colaborador, gestor).

Soluções por tela: telas CRUD, dashboards, restrições.

Persistência: Supabase (ou outra) — multiusuário, não multitenant.

Critérios "pronto quando": ex.: "Tela X pronta quando usuário conseguir criar solicitação com validações A, B e C e notificação ao gestor".

Kit de construção gerado:

docs/PRD.md

docs/specs/fase-1.md

docs/fases (arquivo de controle com checklist)

agents (instruções para agente: ler PRD, trabalhar apenas na primeira fase não concluída, não inventar requisitos)

Workflow: IA propõe fases → gera spec da fase 1 → planeja com ferramenta (ex.: Cursor) → executar → validar critérios → marcar concluído.

Termos Técnicos

Termo	Definição	Contexto de uso
PRD	Documento de Requisitos do Produto que descreve o que e porquê do produto	Fonte de verdade para construção e para geração do kit
Kit de Construção	Pacote com árvore de pastas, arquivo agents, specs e prompts	Gerado a partir do PRD para orientar agentes de IA
Spec / Fase	Arquivo .md que descreve a fatia de trabalho (requisitos, regras, pronto quando)	Unidade de implementação no SDD
Agents	Arquivo com regras/instruções para o agente de IA	Garante comportamento padronizado do agente durante construção
"Pronto quando"	Critério de aceite que define condições de conclusão de uma item/fase	Evita ambiguidade na validação de entrega
As-is / To-be	Estado atual (as-is) e estado desejado (to-be) do processo	Contextualiza o que NÃO deve ser recriado e o que deve ser construído
Alucinação (IA)	Resposta incorreta ou inventada pela IA por falta de contexto ou excesso de demanda	Risco mitigado por fatiamento e instruções claras no agents

Pontos de Atenção e Erros Comuns

Evitar pedir para a IA "construir tudo de uma vez" colando o PRD inteiro e pedindo implementação completa — isso gera sobrecarga e alucinações.

Não pedir para a IA alterar diretamente o PRD para listar fases; gerar novos arquivos de fases/specs separados.

Não numerar as tarefas do planejador como se a ordem fosse fixa sem validar dependências.

Nunca pular os critérios "pronto quando" ou validar apenas por feeling — critérios objetivos são essenciais.

Atualize o PRD quando houver mudanças de necessidade/solução e regenere o kit em seguida.

Resumo (Pontos-chave)

O PRD é a fonte de verdade e deve conter contexto, necessidades, soluções, regras, persistência e critérios de aceite.

O Kit de Construção traduz o PRD em artefatos legíveis por agentes (árvore de pastas, agents, specs, prompts).

Spec Driven Development divide o trabalho em fases (fatias) controladas, reduzindo riscos de alucinações de IA.

O agente deve ter regras claras (arquivo agents): ler PRD, trabalhar só na fase ativa, não inventar requisitos e marcar fases concluídas.

Validação objetiva (pronto quando) é essencial para qualidade e rastreabilidade.

Mantenha PRD e kit versionados e atualize-os quando houver mudanças.

Perguntas de Revisão

Compreensão

O que diferencia o PRD do kit de construção?

Por que não devemos pedir para a IA implementar todo o PRD de uma vez?

Aplicação

Descreva os passos para gerar e usar a spec da Fase 1 a partir do PRD.

Como você configuraria o arquivo agents para garantir que a IA não invente requisitos?

Análise

Quais benefícios o SDD traz em termos de controle de qualidade e mitigação de risco?

Em que situações você poderia justificar a execução paralela de fases? Quais precauções tomaria?

Discussão (nível avançado)

Como integrar testes automatizados e CI/CD ao fluxo de Specs para reforçar os critérios "pronto quando"?

Que métricas e indicadores (do PRD) são mais úteis para avaliar sucesso após entrega (ex.: redução de tempo, taxa de erro)?

Exercícios práticos

1. Pegue um processo simples (ex.: solicitação de reembolso) e esboce um PRD curto com: objetivo, principais personas, 3 necessidades MUST, 2 telas e 2 critérios "pronto quando".
2. A partir do PRD do exercício 1, escreva um prompt que peça à IA para gerar as fases (índice) com critérios "pronto quando" para cada fase.
3. Crie uma spec (arquivo MD) para a fase 1 contendo: necessidade, regras, telas envolvidas, dados persistidos e critérios de aceite.

Pré-requisitos e Conhecimentos Assumidos

Noções básicas de gerenciamento de produto (conceitos de requisitos, escopo).

Familiaridade com Markdown e organização de projetos em pastas.

Conceitos básicos de arquitetura de informação e bases de dados (persistência).

Conhecimento prático de ferramentas de IA/agents (opcional, mas recomendado).

Áreas Potencialmente Confusas e Estratégias para Superá-las

Confusão entre contexto (as-is) e requisitos implementáveis: sempre separar claramente no PRD o que é contexto histórico e o que faz parte do to-be.

Incerteza sobre "quando marcar pronto": padronizar critérios de aceite mensuráveis (ex.: fluxos completos, validações presentes, testes manuais automatizados).

Medo de depender demais da IA: tratar o agente como um executor sob regras; revisar e validar cada etapa manualmente até ganhar confiança no fluxo.

BLOCO 3 · AULA 16

Implementação Orientada por Specs: Ciclo, Estrutura de Arquivos e Boas Práticas

Objetivos de Aprendizagem

Definir o que é um PRD, fases e specs no contexto de desenvolvimento orientado por IA.

Explicar a estrutura de pastas e arquivos recomendada para manter o projeto legível pela IA.

Aplicar o ciclo de trabalho: planejar (spec), implementar, validar e marcar como concluído por fase.

Avaliar quando e como atualizar PRD, fases e specs diante de mudanças de escopo.

Comparar ferramentas e práticas (agents.md x cursor rules) e decidir qual estratégia usar em cada situação.

Conceitos Principais

PRD (Documento de Requisitos do Produto)

Definição: Documento que responde "por que" e "o que" do produto — propósito, objetivos de negócio, escopo alto.

Localização: Deve ficar dentro da pasta do projeto (por exemplo docs/PRD.md) para que a IA possa ler diretamente.

Boas práticas:

Nunca guardar apenas em downloads, Drive isolado ou chat — colocar no repositório.

Atualizar o PRD sempre que houver mudança de escopo ou necessidade nova.

Fases

Definição: Índice de alto nível do projeto — lista sequencial de entregas (uma linha por fase), cada fase com status (pendente, em progresso, feita).

Geração: Pedir para a IA gerar todas as fases de uma vez (arquivo `fases.md`) para ter visão geral.

Uso:

Serve como "sumário" para o projeto.

Reavaliar quando o PRD for atualizado.

Specs (Especificações por Fase)

Definição: Recorte detalhado de cada fase — descrição técnica e tarefas necessárias para implementação.

Nascimento: Criadas uma por vez ao iniciar a construção daquela fase.

Motivo de criar uma por vez:

Evita duplicidade de trabalho entre fases (tarefas da fase 2 que já foram feitas na fase 1).

Permite responder a mudanças contínuas no produto sem gerar artefatos inúteis.

Fluxo por fase: planejar (gerar spec) → implementar → verificar/validar → marcar como concluído.

Agents.md vs cursor rules

agents.md:

Arquivo localizado na raiz do projeto que contém o prompt/orientações do agente, como "antes de começar, leia o PRD".

Deve ser copiado e colocado no repositório para que os agentes leiam e apliquem as regras específicas do projeto.

cursor rules:

Regras globais aplicadas pelo cursor (ambiente), valem para vários projetos.

Podem não ler os agentes locais; por isso, preferir colocar instruções locais em `agents.md`.

Recomenda-se usar `agents.md` para definir comportamento específico do agente para aquele projeto.

Organização de Arquivos e Estrutura Recomendada

Repositório raíz:

docs/PRD.md (PRD exportado)

agents.md (prompt do agente / instruções do projeto)

fases.md (lista de fases com status)

specs/ (pasta contendo specs por fase, por exemplo `spec-01.md`, `spec-02.md`)

src/ (código produzido)

Regra simples: se a IA não consegue ler o arquivo sozinha, ele está no lugar errado. Coloque tudo no repositório do projeto.

Ciclo de Trabalho (por fase)

1. Colocar PRD e agents.md na pasta do projeto.
2. Pedir para a IA gerar todas as fases (fases.md) — só o índice.
3. Para cada fase:
 - Clicar em "planejar" (ou solicitar "gere a spec desta fase").
 - A IA gera a spec (arquivo único para a fase).
 - Revisar/validar o plano da spec.
 - Executar implementação (código/artefato).
 - Verificar e validar entregas.
 - Marcar a fase como concluída no `fases.md`.
4. Repetir para a próxima fase.

Observação: Não peça para a IA escrever todas as specs detalhadas de uma vez; isso reduz flexibilidade e pode gerar sobreposição ou trabalho desperdiçado.

Como Proceder Quando o PRD Muda

Se só a spec de uma fase precisa mudar: atualizar apenas a spec.

Se o escopo mudou (nova necessidade):

Atualize o PRD (`docs/PRD.md`).

Substitua o PRD no repositório.

Peça para a IA revisar o arquivo de fases (`fases.md`) para ver se novas fases precisam ser criadas ou se a ordem/escopo das atuais muda.

Gere a spec nova apenas para as fases afetadas.

Ponto prático: PRD desatualizado vira ficção em poucas semanas; mantenha sempre atualizado.

Relações entre Conceitos

PRD define "o que" e "por quê" (visão de produto).

Fases são o índice do PRD, visão de entrega organizada.

Specs são o detalhamento operacional de cada fase, usadas para construir.

Agents.md define como o agente IA deve interpretar e atuar sobre PRD, fases e specs.

O ciclo (planejar → implementar → validar) se aplica por spec/fase, criando um fluxo iterativo e adaptável a mudanças.

Aplicações e Exemplos Reais

Exemplo 1: Criação de uma tela de cadastro de usuário

PRD: objetivo (capturar nome, e-mail, senha; regra de negócio de validação).

Fase X: "Cadastro de usuário" (status pendente).

Spec da fase X: rotas, validações, banco de dados, testes.

Implementação: código backend + frontend + testes.

Validação: QA e homologação, marca como feito.

Exemplo 2: Integração com WhatsApp adicionada após fase 2 estar pronta

Mudança no PRD: novo canal de comunicação.

Atualização: PRD atualizado e `fases.md` revisado; possivelmente criar nova fase "Integração WhatsApp".

Gerar spec da nova fase e implementar.

Terminologia Técnica

Termo	Definição	Contexto
PRD	Documento de Requisitos do Produto; descreve propósito, objetivos e escopo.	Localizar em docs/PRD.md para leitura automática pela IA.
Fase	Entrada de alto nível no ciclo do projeto; uma linha por fase com status.	Gerada no arquivo fases.md.
Spec	Especificação detalhada de uma fase; orienta a implementação.	Criada por vez para cada fase (ex.: specs/spec-01.md).
agents.md	Arquivo com instruções para o agente IA específico do projeto.	Colocar na raiz do projeto para garantir que agentes leiam.
cursor rules	Regras gerais do ambiente (cursor); aplicam-se a vários projetos.	Podem ignorar agents locais; usar com cuidado.
Ciclo por fase	Planejar → Implementar → Verificar → Marcar como feita	Método iterativo para reduzir retrabalho.

Pontos de Resumo (Takeaways)

Mantenha PRD e agents.md dentro do repositório para que a IA leia automaticamente.

Gere fases (índice) de uma vez, mas gere specs detalhadas apenas quando for construir a fase.

Trabalhe em ciclos por fase para permitir mudanças e evitar duplicidade.

Atualize PRD sempre que houver mudança de escopo; revise fases em consequência.

Prefira agents.md para regras específicas do projeto ao invés de confiar apenas em cursor rules globais.

Entender o conceito é mais importante do que a ferramenta; implemente o ciclo com a ferramenta que você domina.

Questões de Revisão e Exercícios

Compreensão

1. Defina PRD, fase e spec. Como eles se relacionam?
2. Por que é recomendado criar specs uma por vez em vez de todas de uma vez?

Aplicação

3. Dado um PRD com três features (autenticação, perfil, notificações), descreva uma possível lista de fases (fases.md) com justificativa da ordem.
4. Você recebeu uma nova necessidade: envio de mensagens por WhatsApp. Explique passo a passo o que deve ser atualizado nos arquivos do projeto.

Análise / Discussão

5. Compare as vantagens e desvantagens de manter regras em `agents.md` versus `cursor rules`.
6. Discuta um cenário em que gerar todas as specs de uma vez faria sentido. Quais precauções tomar?

Tarefa prática

7. Monte a estrutura de pastas e arquivos mínima para um novo projeto (lista de comandos ou instruções para pedir à IA criar os arquivos).
8. Simule (escreva) o conteúdo inicial de um `agents.md` com instruções básicas: "Antes de começar, leia o PRD em docs/PRD.md; gere apenas as fases; crie specs uma por vez; sempre validar antes de implementar."

Pré-requisitos e Conhecimentos Assumidos

Familiaridade básica com conceitos de desenvolvimento de software (repositório, arquivos, fases/entregas).

Entendimento mínimo de como interagir com ferramentas de IA que leem repositórios (agents, cursor).

Noções de versionamento (substituir arquivos no repositório quando atualizar PRD).

Possíveis Dúvidas e Equívocos Comuns

"Preciso colocar tudo no chat para a IA ler" — não; prefira colocar os arquivos no repositório para leitura automática.

"Gerar todas as specs de uma vez é sempre melhor" — não; isso costuma gerar retrabalho e falta de adaptabilidade.

"cursor rules é sempre suficiente" — não necessariamente; regras globais podem ignorar nuances do projeto; use agents.md para instruções locais.

Confusão entre "fase" e "spec": fase = nome/entrega; spec = detalhamento de como implementar essa entrega.

BLOCO 3 • AULA 17

Projetos Curtos: Como Planejar um PRD Enxuto e Executável

Objetivos de Aprendizagem

Definir o que é um PRD (Product Requirements Document) e explicar sua importância em projetos curtos.

Analisar quando usar um PRD completo vs. um PRD enxuto.

Aplicar técnicas rápidas (mapa mental, áudio, IA First) para gerar um PRD curto e acionável.

Avaliar riscos de escopo e identificar estratégias para conter a expansão indesejada.

Criar um briefing mínimo (PRD curto) e dividir o projeto em fases executáveis.

Conceitos Principais

1. PRD (Documento de Requisitos do Produto) para projetos curtos

Explicação: Mesmo projetos pequenos se beneficiam de um PRD — ele cria um acordo sobre o que será entregue e o que ficará de fora.

Aplicação prática: Use um PRD simples quando o projeto for pessoal, uma automação rápida ou uma entrega no final de semana.

Vantagem: Menos retrabalho, menor chance de abandonar o projeto por falta de planejamento.

2. Quando usar PRD enxuto vs. PRD completo

PRD completo: necessário para sistemas grandes, com múltiplas fases, integridade de dados e várias partes interessadas.

PRD enxuto: ideal para soluções pequenas; foca nas necessidades essenciais, critérios de pronto e fases principais.

Regra prática: "Mega quando precisa, PRD sempre." Se escopo explodir, migre para o processo completo.

3. Técnicas rápidas de estruturação

Mapa mental:

Centro: problema.

Ramificações: quem sofre, passos atuais, dores, o que a solução precisa, o que fica de fora, telas e histórias da tela.

Benefício: visão em zoom out (visão de drone) para evitar perder detalhes importantes ao trabalhar isoladamente.

Uso de áudio:

Grave suas ideias em voz (ex.: Handy).

Transcreva o áudio e use como insumo para a IA gerar um PRD curto.

Vantagem: comunicação mais rápida do raciocínio e menor atrito para extrair requisitos iniciais.

IA First:

Se não souber a solução, peça opções à IA (por exemplo: "Me dê 5 soluções possíveis").

Escolha a que fizer mais sentido e registre essa decisão no PRD.

4. Estrutura mínima recomendada de um PRD curto

Problemas (dor): O que é o problema central que você resolve?

Objetivos: O que você quer alcançar com a solução?

Pessoas/Usuários: Quem usará? Onde vivem os dados?

Escopo: O que entra e o que fica de fora (limitações da Fase 1).

Necessidades (3–5 itens): Lista de necessidades essenciais.

Telas/Histórias: Ideias de telas e o que cada tela precisa permitir (histórias da tela).

Regras e critérios de pronto: Regras específicas e condições que definem "pronto".

Ciclo: Fluxo básico de início ao fim (ex.: começa aqui, termina ali).

Fases: Divisão em 1–3 fases executáveis para projetos curtos.

5. Cuidados e boas práticas

Não aceitar escopo inventado pela IA: corte imediatamente itens que não fazem parte do objetivo.

Se o escopo expandir muito: voltar ao planejador completo.

Não começar sem PRD (mesmo que seja curto).

Não pedir tudo de uma vez: gere fases e execute fase a fase.

Decidir persistência dos dados e primeiro usuário (quem será o usuário inicial).

Guardar áudios e transcrições junto com o PRD para histórico e iteração.

Terminologia Técnica

Termo	Definição	Contexto
PRD (Documento de Requisitos do Produto)	Documento que descreve problemas, objetivos, usuários, requisitos e critérios de aceite de um produto	Usado para alinhar expectativas antes de desenvolver, mesmo em projetos curtos
IA First	Abordagem que pede ajuda à inteligência artificial para gerar ideias e soluções	Útil quando não se tem clareza sobre alternativas de solução
Mapa mental	Diagrama hierárquico que organiza ideias ao redor de um núcleo	Ferramenta para visão abrangente (zoom out) do problema e solução
Critério de pronto	Condição mensurável que determina quando uma tarefa/produto está concluído	Ajuda a evitar entregas ambíguas e retrabalhos
Fase	Segmento do projeto com objetivos executáveis e entregáveis	Divide projetos pequenos em passos gerenciáveis

Resumo dos Pontos-Chave

Mesmo projetos pequenos precisam de PRD; prefira um PRD enxuto e executável.

Use mapa mental e áudio para capturar rapidamente o problema e as necessidades.

A IA é uma aliada: peça soluções quando estiver indeciso, mas controle o escopo.

Defina claramente o que fica FORA na Fase 1 para reduzir complexidade e ansiedade.

Separe o projeto em fases e peça à IA para gerar planejamento fase por fase, se necessário.

Documente áudios e transcrições; eles servem como insumo para futuras iterações.

Evite começar a construir sem PRD — é uma das principais causas de retrabalho e abandono.

Perguntas de Revisão e Prática

Compreensão

1. Por que mesmo um projeto simples deve ter um PRD? Responda em 2–3 frases.
2. Quais são os itens essenciais de um PRD curto? Liste pelo menos cinco.

Aplicação

3. Você tem a ideia de um "gerador de imagens para postagens". Descreva rapidamente (em mapa mental resumido) o problema, três dores do usuário e duas coisas que devem ficar fora na Fase 1.
4. Grave um áudio de 1–2 minutos explicando o problema central do seu projeto e transcreva. Use a transcrição como base para pedir à IA um PRD curto.

Análise

5. Dê um exemplo de como um escopo mal controlado poderia arruinar um projeto curto e explique como a estratégia "O que fica de fora" evita isso.
6. Compare PRD enxuto x PRD completo: quando migrar de um para o outro?

Discussão

7. Debata (em pares ou grupo) se o uso intenso de IA para gerar requisitos pode levar a soluções padronizadas demais. Quais contramedidas adotar?

Desafio (avançado)

8. Pegue um projeto pessoal que você já abandonou no passado. Reescreva um PRD curto seguindo a estrutura apresentada (problema, objetivos, usuários, 3–5 necessidades, o que fica de fora, critérios de pronto, fases). Avalie o quanto isso teria reduzido retrabalho.

Pré-requisitos e Conhecimento Assumido

Noções básicas de desenvolvimento de software e ciclo de projeto (entendimento de termos como "entregável" e "fase").

Familiaridade com ferramentas simples de produtividade (mapa mental, gravação de áudio, editores de texto).

Conhecimento elementar sobre como interagir com ferramentas de IA (ex.: prompts, pedir iterações).

Potenciais Áreas de Confusão e Como Evitá-las

"PRD curto" não é sinônimo de sem documentação: ainda exige critérios de pronto e escopo bem definidos.

Confiar cegamente na IA: ela pode propor funcionalidades irrelevantes. Sempre valide contra objetivos claros.

Fases vs. Specs detalhadas: nem sempre precisa de especificações técnicas para projetos curtos, mas as fases devem ser claras e executáveis.

Persistência e primeiro usuário: não definir pode levar a um produto sem público-alvo ou sem estratégia de dados.

Modelo Prático: Briefing Rápido de PRD (Template)

Problema central:

Objetivo:

Usuário primário (primeiro usuário):

O que entra (escopo):

O que fica de fora:

3–5 necessidades essenciais:

Telas/Historias principais:

Regras importantes:

Critério de pronto:

Fases sugeridas (Fase 1, Fase 2, Fase 3):

Observações/Áudio e transcrição anexos:

Use esse template para criar PRDs curtos e, se necessário, peça à IA para expandir cada item em tarefas executáveis.

Fim do material. Comece praticando com um micro-projeto (ex.: gerador de imagens para uma rede social) e construa um PRD curto seguindo o template acima. Boa sorte e evite o retrabalho!

BLOCO 3 • AULA 18

Planejamento de Projetos com IA — Guia de Estudo

Objetivos de Aprendizagem

Definir os conceitos fundamentais do planejamento de projetos com foco em IA.

Identificar e descrever as etapas do ciclo: as-is → planejamento → to-be → execução faseada pela IA.

Analisar riscos, necessidades e prioridades para construir um PRD (Product Requirements Document) eficiente.

Aplicar boas práticas para estruturar projetos, organizar documentação e evitar erros comuns.

Avaliar estratégias de integração entre ferramentas de IA e fluxos humanos para validar planos e aumentar produtividade.

Conceitos Principais

1. Profissionalização do Planejamento com IA

Explicação: Conhecer e aplicar metodologia de planejamento diferencia profissionais que "sabem usar IA" daqueles que "constroem com IA". Planejar inclui entrevistas, identificação de riscos, definição de prioridades (must vs nice-to-have) e comunicação clara com stakeholders.

Aplicação prática: Ao iniciar um projeto, realize uma entrevista com o cliente/gestor para mapear dores, riscos, regras e responsáveis antes de tocar qualquer ferramenta.

2. Jornada: as-is → Planejamento → to-be → Execução

as-is: Mapear o estado atual do processo (fluxos, entradas/saídas, pontos de atrito).

Planejamento: Definir escopo, responsabilidades, regras de negócio e necessidades de persistência dos dados.

to-be: Projetar a solução desejada (telas, integrações, funcionalidades) — o produto final proposto.

Execução faseada: Construir por fases (sprints) e entregar incrementalmente. Não mandar construir todo o PRD de uma vez.

Exemplo de caso: automação de férias

Mapear processo atual: quem pede, quem aprova, documentos necessários.

Definir regras: permissões do gestor, notificações ao colaborador, validações.

Necessidades: local seguro para armazenar solicitações; indicador de sucesso (ex.: redução de tempo de aprovação).

Solução (to-be): tela de solicitação, integração com base de dados (Supabase), notificações para gestor.

3. PRD (Product Requirements Document) e Estruturação de Projeto

PRD: documento organizado explicando necessidades, riscos, escopo, regras, indicadores e persistência de dados.

Pasta de projeto: docs/ com PRD.md, agentes.md, fases e arquivos relacionados.

Fluxo recomendado: gerar PRD → validar → construir fase por fase via IA.

Boas práticas:

Faça o PRD antes de construir.

Salve o PRD em repositório (arquivo .md) e não apenas em chat de IA.

Estruture a pasta do projeto e salve JSONs/exportações do planejador.

Use checklist para validar itens obrigatórios.

4. Necessidades vs Soluções

Necessidades: o que precisa existir (ex.: colaborador precisa ter um formulário para cadastrar).

Soluções: como essas necessidades serão atendidas (ex.: tela, API, banco).

Indicadores: métricas para avaliar sucesso (ex.: tempo médio de aprovação, taxa de erros).

5. Erros Comuns no Planejamento

Começar pela ferramenta sem mapear o processo.
Construir sem PRD ou com PRD mal definido.
Escrever PRD manualmente sem usar IA quando apropriado.
Confundir as-is (estado atual) com to-be (estado futuro).
Entregar todo o PRD para construir de uma vez.
Descrever detalhes técnicos do banco (campos) em vez de contar a história do fluxo.
Não decidir onde os dados vão viver (banco, CSV, API).
Confundir multi-usuário (múltiplos usuarios na mesma empresa) com multi-tenant (múltiplas empresas).
Esquecer definição do primeiro usuário para login.
Numerar fases manualmente em vez de deixar a IA propor (quando apropriado).
Deixar o PRD fora do repositório.
Criar MVP inchado (muitas features sem priorização).
Pular o modo de construção do PRD na base (não usar o sistema disponível).

6. Boas Práticas Recomendadas

Estrutura primeiro: você define problema, regras, escopo e persistência; a IA ajuda a construir.
Peça o PRD para a IA e valide fase por fase.
Planeje aquilo que só você sabe (regra de negócio e contexto).
Escolha modo de construção coerente com as necessidades (ex.: tela + planilha coerentes).
Confirme fases antes de executar.
Salve JSONs e PRD no planejador/repositório.
Use múltiplas IAs para validar e revisar (ex.: uma cria, outra valida), aproveitando pontos fortes diferentes.
Automatize checklists e mantenha status atualizados (ajuste manual quando necessário).

Terminologia Técnica

Termo	Definição	Contexto
as-is	Estado atual do processo ou produto.	Primeiro passo do mapeamento; descreve entradas/saídas e atritos.
to-be	Estado desejado ou solução proposta.	Representa telas, integrações e funcionalidades planejadas.
PRD	Documento de requisitos do produto (Product Requirements Document).	Documento central que descreve necessidades, regras, indicadores e persistência.
Persistência de dados	Onde e como os dados serão armazenados (banco, Supabase, CSV, API).	Decisão crítica que não deve ser delegada totalmente à IA.
Multi-usuário	Vários usuários da mesma organização utilizando o sistema.	Cenário interno (ex.: RH e colaboradores da mesma empresa).
Multi-tenant	Múltiplas empresas (tenants) usando a mesma aplicação, isoladas entre si.	Cenário SaaS para clientes distintos.
MVP inchado	Produto mínimo viável com escopo excessivo.	Risco de atrasos e complexidade; priorizar faz parte do planejamento.
Checklists	Lista de verificação de itens obrigatórios do planejamento.	Ferramenta de controle de qualidade do PRD e das fases.

Pontos de Resumo

Planejamento é diferencial competitivo: confere senioridade, respeito e melhores projetos/remuneração.

Nunca comece a codar sem PRD; sempre privilegie validação faseada.

Decisões de persistência e escopo são suas (do planejador/produto), a IA auxilia na execução.

Use ferramentas de IA para acelerar, mas mantenha controle sobre regras de negócio e estrutura.

Valide planos com múltiplas IAs quando possível para reduzir carga mental e melhorar qualidade.

Salve artefatos (PRD.md, JSON do planejador) no repositório do projeto.

Perguntas de Revisão

Compreensão

1. Defina em suas palavras o que é um PRD e por que ele é essencial antes de começar a construir.
2. Quais as diferenças entre as-is e to-be? Dê um exemplo prático em um fluxo de férias.

Aplicação

3. Imagine um processo de solicitação de reembolso: liste as três principais necessidades que deveriam constar no PRD.
4. Você está planejando um sistema SaaS. Como decidiria entre design multi-usuário e multi-tenant? Quais implicações técnicas essa escolha traz?

Análise

5. Quais riscos você identificaria ao delegar totalmente a definição de persistência de dados para uma IA?
6. Analise um PRD hipotético que inclui 20 funcionalidades na primeira entrega. Quais passos você faria para evitar um MVP inchado?

Discussão / Reflexão

7. Discuta os prós e contras de usar duas IAs (uma para criar, outra para validar) no fluxo de planejamento.
8. Como o uso de checklists melhora a confiabilidade do planejamento em equipe?

Desafios (nível avançado)

9. Projete um checklist mínimo (5 itens) para o planejador antes de enviar uma fase para construção pela IA.
10. Dê um roteiro (passo a passo) para transformar uma necessidade de negócio em uma primeira fase executável pela IA.

Respostas Sugeridas (Resumos)

1. PRD é o documento que descreve necessidades, regras, indicadores e persistência — serve como contrato entre produto, engenharia e stakeholders.
2. as-is descreve como o processo funciona hoje; to-be descreve como deverá funcionar. Ex.: as-is: pedido de férias por e-mail; to-be: formulário online com aprovação automática por gestor.
3. Exemplos: formulário de solicitação seguro; armazenamento das solicitações em banco seguro; notificação ao gestor com histórico.
4. Multi-tenant se escolhe quando o produto será vendido a várias empresas com isolamento; traz requisitos de segurança, separação de dados e configuração por tenant. Multi-usuário serve quando múltiplos usuários da mesma empresa acessam o sistema.
5. Risco: a IA pode sugerir uma solução inconsistente ou ineficiente para sua infraestrutura; perda de controle sobre conformidade, segurança e custos.
6. Priorizar funcionalidades por valor/impacto/risco; dividir em releases; criar MVP mínimo; validar com stakeholders.

Áreas de Dificuldade e Estratégias de Superação

Dificuldade: decidir a persistência de dados.

Estratégia: mapear requisitos de performance, segurança e conformidade; escolher entre Supabase, bancos relacionais, APIs ou arquivos conforme restrições.

Dificuldade: priorização de features.

Estratégia: usar matriz impacto/esforço e definir must-have vs nice-to-have; envolver stakeholders para alinhamento.

Dificuldade: controlar a produção documental (muitos arquivos e versões).

Estratégia: padronizar repositório (docs/), versionar PRD.md e salvar JSONs exportados do planejador.

Exemplo Prático: Checklists e Fluxo Mínimo para uma Fase

Checklist mínimo antes de enviar fase para construção:

1. PRD.md atualizado com necessidade, regras e indicador de sucesso.
2. Decisão tomada sobre persistência de dados (ex.: Supabase).
3. Responsáveis definidos (quem é gestor, quem é executor).
4. Arquivo da fase salvo em docs/ e exportado em JSON.
5. Checklist de segurança básico preenchido (ex.: controle de acesso, primeiro usuário definido).

Fluxo passo a passo (da necessidade à primeira fase executável):

1. Entrevista com stakeholder → mapear dor e contexto.
2. Registrar as-is e identificar atritos.
3. Definir regras, responsáveis e indicadores.
4. Escolher persistência de dados e plataforma (ex.: Supabase).
5. Escrever PRD resumido (necessidades + regras + indicador).
6. Pedir à IA para gerar proposta de fases com base no PRD.
7. Validar fase 1 (manter enxuto, com valor claro).
8. Salvar arquivos (PRD.md, fase1.md, JSON) em docs/.
9. Enviar fase 1 para construção incremental pela IA.
10. Validar entregável, coletar feedback e iterar.